A VERSION-AWARE RAG FOR RELIABLE NETWORK CLI CONFIGURATION SYNTHESIS

A PROJECT REPORT

Submitted to

Amrita Vishwa Vidyapeetham

in partial fulfilment for the award of the degree of

BACHELORS OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

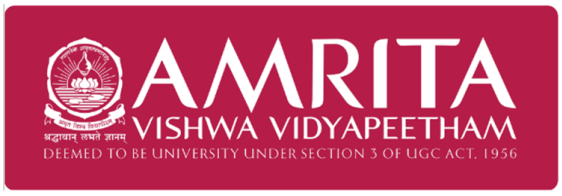
By

ARYAN MADHUSUDHANAN
(Reg. No. CH.SC.U4CSE23207)

SRIVISHNU T
(Reg. No. CH.SC.U4CSE23249)

Supervisor

Mrs. V Nivethitha



AMRITA VISHWA VIDYAPEETHAM
AMRITA SCHOOL OF COMPUTING
CHENNAI - 601103

April 2026

BONAFIDE CERTIFICATE

This is to certify that this project report entitled **"A VERSION-AWARE RAG FOR RELIABLE NETWORK CLI CONFIGURATION SYNTHESIS"** is the bonafide work of **"Aryan Madhusudhanan (CH.SC.U4CSE23207) and Srivishnu T (CH.SC.U4CSE23249)"** who carried out the project work under my supervision.

SIGNATURE

Dr. S. BAGHAVATHI PRIYA
CHAIRPERSON
Department of CSE
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Chennai - 601103

SIGNATURE

Mrs. V NIVETHITHA
SUPERVISOR
Department of CSE
Amrita School of Computing
Amrita Vishwa Vidyapeetham
Chennai - 601103

INTERNAL EXAMINER

INTERNAL EXAMINER

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

SCHOOL OF COMPUTING CHENNAI

DECLARATION BY THE CANDIDATE

We hereby declare that the project report entitled **"A VERSION-AWARE RAG FOR RELIABLE NETWORK CLI CONFIGURATION SYNTHESIS"** is a bona fide record of the Phase 1 work done by us under the guidance of our supervisor. This work has not formed the basis for the award of any degree, diploma, associateship, fellowship, or other similar title to any candidate of any University.

SIGNATURE

Aryan Madhusudhanan & Srivishnu T

(CH.SC.U4CSE23207) & (CH.SC.U4CSE23249)

ABSTRACT

The combination of Large Language Models with Intent-Based Networking systems enables organizations to achieve their operational goals through remote operations. The use of generalist LLMs results in Syntactic Fabrication and Version Pollution problems because the system combines commands from different operating systems which results in failed configuration attempts. The research introduces a Version-Aware Retrieval-Augmented Generation (RAG) architecture that enables accurate CLI generation across different systems which includes Cisco IOS-XE and Juniper JunOS. The system employs Hierarchical Knowledge Grounding which restricts LLM generation to vendor-provided documents that filter content through metadata and tie it to specific operating system versions. The system includes a Syntactic Verification Gate which uses Batfish static analysis to identify errors that would occur during deployment. The evaluation of multi-vendor intent benchmarks shows that version-locked grounding achieves zero hallucinations (0% vs. 4.3% baseline) and an 87.0% Syntactic Validity rate. The study shows that 17.4% of Critical Failure Rate (CFR) errors remain which include hidden semantic mistakes that pass syntax tests but do not achieve operational objectives. The system creates a Grounding Audit Trail which establishes a 49,000:1 cognitive compression ratio that allows human reviewers to spend less time searching through documentation. The research indicates that AI-based synthesis systems cannot operate without human control which leads to the implementation of an Audit-Ready framework that improves expert human monitoring.

Keywords: *IBN* , *LLMs* , *RAG* , *CLI* , *Hallucination* , *Batfish* , *Verification*

ACKNOWLEDGEMENT

This project work would not have been possible without the contribution of many people. It gives me immense pleasure to express my profound gratitude to our honorable Chancellor **Sri Mata Amritanandamayi Devi**, for her blessings and for being a source of inspiration. I am indebted to extend our gratitude to our Administrative Director, **Swami Vinayamritananda Puri** and Campus Director, **Mr. I B Manikantan** Amrita School of Computing and Engineering, for facilitating us all the facilities and extended support to gain valuable education and learning experience.

I register my special thanks to **Dr. V. Jayakumar**, Principal Amrita School of Computing and Engineering for the support given to me in the successful conduct of this project. I wish to express my sincere gratitude to **Dr. S. Baghavathi Priya**, Chairperson, **Dr. S. Sountharrajan**, Associate Professor, **Dr. S. Natarajan**, Program Chair and to my supervisor **Mrs. V Nivethitha**, Department of Computer Science and Engineering for their inspiring guidance, personal involvement and constant encouragement during the entire course of this work.

I am grateful to Project Coordinator, Review Panel Members and the entire faculty of the Department of Computer Science & Engineering, for their constructive criticisms and valuable suggestions which have been a rich source to improve the quality of this work.

Aryan Madhusudhanan
(CH.SC.U4CSE23207)

Srivishnu T
(CH.SC.U4CSE23249)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	Abstract	iv
	Acknowledgement	v
	List of Tables	ix
	List of Figures	x
	List of Symbols and Abbreviations	xi
1	**INTRODUCTION**	1
	1.1 BACKGROUND: THE EVOLUTION OF NETWORK MANAGEMENT	1
	1.2 MOTIVATION	1
	1.3 ROLE OF RAG IN IBN	1
	1.4 THE IBN PARADIGM AND THE NORTHBOUND CHALLENGE	2
	1.5 OVERVIEW OF THE PROPOSED SYSTEM: ADDRESSING THE RELIABILITY CRISIS	2
	1.6 PHASE-WISE RESEARCH OBJECTIVES	3
	1.7 SIGNIFICANCE OF THE PROJECT	4
	1.8 SCOPE OF THE STUDY	4
2	**LITERATURE REVIEW**	6
	2.1 OVERVIEW	6
	2.2 TAXONOMY OF HISTORICAL EVOLUTION IN NETWORK AUTOMATION	6
	2.3 REVIEW OF PREVIOUS PAPERS	6

	2.4 LITERATURE SUMMARY TABLE	18
	2.5 SUMMARY OF INDIVIDUAL STUDIES	21
	2.6 SYNTHESIS OF TECHNICAL IMPEDIMENTS IN CURRENT PARADIGMS	31
3	**PROBLEM STATEMENT AND METHODOLOGY**	33
	3.1 PROBLEM STATEMENT	33
	3.2 OVERVIEW OF THE PROPOSED SYSTEM	33
	3.3 DATASET DESCRIPTION	34
	3.4 DATASET COMPOSITION	34
	3.5 DATA PREPROCESSING	35
4	**SYSTEM DESIGN AND EXPERIMENTAL WORK**	36
	4.1 SOFTWARE REQUIREMENTS	36
	4.2 SYSTEM ARCHITECTURE OVERVIEW	36
	4.3 STAGE 1: INTENT PARSING (SEMANTIC ABSTRACTION)	36
	4.4 STAGE 2: VERSION-AWARE RAG (METADATA-ANCHORED RETRIEVAL)	37
	4.5 STAGE 3: SYNTHESIS AND GROUNDING	37
	4.6 STAGE 4: BATFISH VERIFICATION GATE	37
	4.7 INTENT PARSER IMPLEMENTATION	38
	4.8 RETRIEVAL ALGORITHM IMPLEMENTATION	38
	4.9 EVALUATION METRICS	39
	4.10 TESTING STRATEGY	40
	4.11 EXPERIMENTAL WORK AND SIMULATION ENVIRONMENT	40
5	**RESULTS AND DISCUSSIONS**	41

5.1 PERFORMANCE METRICS 41

5.2 EXPERIMENTAL SETUP 41

5.3 QUANTITATIVE PERFORMANCE ANALYSIS 41

5.4 ANALYSIS OF HALLUCINATION REDUCTION 42

5.5 SYNTACTIC VALIDITY VIA BATFISH VALIDATOR 43

5.6 ANALYSIS OF SEMANTIC FAITHFULNESS 43

5.7 SAMPLE SYSTEM OUTPUT: END-TO-END EXAMPLE 43

5.8 CASE STUDY: MULTI-VENDOR OSPF SYNTHESIS 44

5.9 CRITICAL FAILURE RATE AND THE SEMANTIC GAP 46

5.10 QUALITATIVE ANALYSIS: THE GROUNDING AUDIT TRAIL 46

5.11 DISCUSSION OF PERFORMANCE IMPROVEMENTS AND REMAINING GAPS 46

5.12 STATISTICAL SIGNIFICANCE ANALYSIS 47

5.13 ERROR DISTRIBUTION ANALYSIS 48

5.14 ABLATION STUDY: COMPONENT-WISE CONTRIBUTION 48

5.15 KNOWLEDGE BASE COVERAGE ANALYSIS 49

5.16 ERROR CASE ANALYSIS: HALLUCINATION DETECTION EXAMPLES 50

5.17 RETRIEVAL PRECISION ANALYSIS 50

5.18 SYSTEM LATENCY ANALYSIS 51

6 CONCLUSIONS AND SCOPE FOR FURTHER WORK 52

6.1 CONCLUSION 52

6.2 FUTURE WORK 53

REFERENCES 54

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	LITERATURE SUMMARY TABLE	18
5.1	COMPARATIVE PERFORMANCE RESULTS	42
5.2	EXTRACTED INTENT PARAMETERS	44
5.3	RETRIEVED DOCUMENTATION CHUNKS	44
5.4	STATISTICAL SIGNIFICANCE OF PERFORMANCE IMPROVEMENTS	47
5.5	SYNTACTIC FAILURE DISTRIBUTION	48
5.6	ABLATION STUDY: COMPONENT-WISE PERFORMANCE CONTRIBUTION	48
5.7	KNOWLEDGE BASE COVERAGE BY PROTOCOL CATEGORY	49
5.8	END-TO-END PROCESSING LATENCY BREAKDOWN	51

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	ARCHITECTURE DIAGRAM OF THE PROPOSED SYSTEM	36
4.2	STRUCTUREDINTENT PYDANTIC MODEL SCHEMA	38
4.3	COMPARISON OF DIFFERENT CHUNKING STRATEGIES SHOWING THAT TIERED HIERARCHICAL CHUNKING MINIMIZES LLM HALLUCINATIONS BY PRESERVING CONTEXT.	40
5.1	COMPARATIVE ANALYSIS ACROSS FOUR KEY RELIABILITY METRICS. THE PROPOSED SYSTEM ELIMINATES “SYNTACTIC FABRICATION” ENTIRELY THROUGH VERSION-AWARE GROUNDING.	42
5.2	GENERATED OSPF CONFIGURATION	44
5.3	MULTI-VENDOR OSPF INTENT	45
5.4	GENERATED CISCO IOS-XE 17.6 CONFIGURATION	45
5.5	GENERATED JUNIPER JUNOS 22.4 CONFIGURATION	45

LIST OF SYMBOLS AND ABBREVIATIONS

ACL	-	Access Control List
API	-	Application Programming Interface
AS	-	Autonomous System
BGP	-	Border Gateway Protocol
CCR	-	Cognitive Compression Ratio
CFR	-	Critical Failure Rate
CLI	-	Command Line Interface
HITL	-	Human-in-the-Loop
HR	-	Hallucination Rate
IBN	-	Intent-Based Networking
IOS-XE	-	Internetwork Operating System (Cisco)
IP	-	Internet Protocol
JSON	-	JavaScript Object Notation
JunOS	-	Juniper Network Operating System
LLM	-	Large Language Model
MD5	-	Message Digest 5
MTU	-	Maximum Transmission Unit
NLP	-	Natural Language Processing
NLU	-	Natural Language Understanding
OS	-	Operating System
OSPF	-	Open Shortest Path First
RAG	-	Retrieval-Augmented Generation
RC	-	Review Coverage
RP	-	Retrieval Precision
SOTA	-	State-of-the-Art
SV	-	Syntactic Validity
VLAN	-	Virtual Local Area Network

CHAPTER 1

INTRODUCTION

1.1 Background: The Evolution of Network Management

Network management has evolved from its past where it was imperative as well as manual CLI interactions to an abstract, declarative model. Human operators were doing all network operations at the time, which was discovered to be risky due to errors and incapable of meeting requirements from large networks with different systems. Intent-Based Networking (IBN) is a new construction of networking, allowing for automatic conversion of high-level natural language goals into configurations of network devices. New sets of functionality built into modern IBN systems have come about because they utilize LLMs and SLMs to read unstructured, human language. The system has also adopted Retrieval-Augmented Generation (RAG) to build reliable AI translation by means of vendor documentation that enhances its translation precision.

1.2 Motivation

Generalist LLM models have potential but they are black boxes that cause hallucinations and negative transfer when applied in telecommunications. The general rules of linguistics are incapable of describing the rigorous principles according to which the network logic and syntax operate. The current RAG-based systems do not support version-specific requirements that leads to Version pollution since they combine commands between OS releases. The operators are not able to apply AI-generated configurations to production systems since the system does not have a definite syntactic validation and full audit trail validation. This reliability issue presents the biggest challenge to autonomous network synthesis.

1.3 Role of RAG in IBN

The project proposes a Version-Aware RAG architecture in order to provide a fully-realized architecture linking the high-level intent to deployable configuration. The system attains precise

Command Line Interface standard scripts via its rigid LLM generation architecture that has version-bound vendor manuals as its foundation. The framework relies on a conversational feedback cycle to address semantic errors prior to it producing output and it relies on deterministic verification systems such as Batfish to check interoperability between systems across vendors and syntactical correctness. The system develops an open documentation system that gives full traceability of audits which reduces hallucinations as it develops operator trust when situations of critical network arise.

1.4 The IBN Paradigm and the Northbound Challenge

The next step in the development of this is Intent-Based Networking (IBN). The operators have their objectives in business that are defined at high level to make their operating processes. The main problem of IBN is in its central part of translation. Transforming unstructured natural language to network operating system (NOS) syntax can only be done with a full comprehension of the meaning of the words and technicality of vendor systems.

1.5 Overview of the Proposed System: Addressing the Reliability Crisis

The proposed system is a reliability-first synthesis engine that addresses the so-called Northbound gap and removes the fallibility that is inherent in generalist LLMs. Two main forms of failure are detected and addressed in the architecture [10], [23]:

1. **Syntactic Fabrication**: Where the model generates plausible but non-existent keywords or parameters that do not exist in the vendor's grammar.
2. **Version Pollution**: Where the model erroneously retrieves valid syntax from one OS release (e.g., Cisco IOS 15.x) and applies it to another (e.g., IOS-XE 17.6), leading to configuration rejection.

By implementing a **Version-Aware Retrieval-Augmented Generation (RAG)** pipeline, our system locks the LLM's knowledge grounding to specific vendor firmware releases. This is followed by a deterministic **Syntactic Verification Gate** using Batfish static analysis, ensuring that only grammatically correct and logically sound configurations are presented to the operator.

1.6 Phase-Wise Research Objectives

This project will be separated into three stages of the development lifecycle to mediate between an inactive configuration synthesis and a fully automated / self healing network ecosystem.

1.6.1 Phase 1: Knowledge Grounding and Syntactic Synthesis (Current Work)

The primary purpose of this process is to eliminate the probabilistic fallibility of LLMs through retrieval based on documentation:

- **Objective 1: Develop a Version-Aware Knowledge Base**: Building a high-fidelity vector store of 10,000+ chunks from official Cisco and Juniper documentation with strict OS version metadata tags.
- **Objective 2: Implement Hierarchical Knowledge Grounding**: Ensure that LLM (Llama 3.1 8B) outputs CLI commands only based on retrieved, versioned context
- **Objective 3: Establish a Deterministic Verification Pipeline**: Using Batfish static analysis to detect syntax errors and "off-by-one" parameter inputs prior to deployment
- **Objective 4: Provide a Grounding Audit Trail**: Metrics evaluating all Syntactic Validity (SV), Hallucination Rate (HR) and Cognitive Compression Ratio (CCR) with command level citations

1.6.2 Phase 2: Digital Twin Integration and Logical Verification (Proposed)

The second phase turns from syntax to semantics to operating state:

- **Objective 1: Digital Twin Realization**: Creating a virtualized representation of the target production topology with **GNS3** or **Eve-NG**.
- **Objective 2: Runtime Logical Validation**: Used synthesized CLI in virtualized topologies to validate operational states and protocol stability.
- **Objective 3: State-Based Correction**: The identification of "silent" semantic errors like routing loops that static analysis cannot catch.
- **Objective 4: Behavioral Compliance**: Verifying that GNS3 behaves as intended by checking that outcomes match the original intent.

1.6.3 Phase 3: Closed-Loop Automation and Self-Healing (Proposed)

The last phase target is for lifecycle management to take place autonomously:

- **Objective 1: Structured Log Parsing**: Creating a Structured Log Parser to convert raw GNS3/Batfish error logs into digest-able feedback for the LLM
- **Objective 2: Contextual Retry Mechanism**: Backing some failure reason to the engine for focused “patching” using a contextual retry mechanism.
- **Objective 3: Prompt Refinement Optimization**: Maximizing immediate refinement to direct the LLM in the correction of individual configuration blocks eliminating complete re-synthesis.
- **Objective 4: Critical Failure Rate Reduction**: Decrease of the Critical Failure Rate (CFR) of 17.4% to 0% via the process of refinement, which is to be done repeatedly on a feedback basis [16], [27].

1.7 Significance of the Project

The project is of great importance to the academic research as well as the industry network operations:

- **Operational Safety**: With a synthesized command checking system, the system enables human operators to confirm what they have synthesized with a formal page of the manuals, a factor that alleviates the cognitive burden of documentation search by 49,000 times.
- **Vendor Neutrality**: The modular architecture is created to accommodate the heterogeneous environment (Cisco, Juniper, etc.) to make it easier to have the multivendor data centers managed.
- **Cost Reduction**: The translation of the “Northbound” is made automatic so that the time spent in the configuration deployment has been decreased by hours to seconds and the risk of an outage, due to human mistakes, is reduced to minimum costs.[13].

1.8 Scope of the Study

This study concentrates on the third IBN lifecycle stage, which is the Synthesis and Verification stage. This study will be limited to:

- **Target OS Platforms**: Cisco IOS-XE (Release 17.6) and Juniper JunOS (Release 22.4).

- **Network Protocols**: OSPF (Open Shortest Path First) and BGP (Border Gateway Protocol) routing facilities A VLAN administrative, and standard/extended ACLs.
- **Toolchain**: Generation is done with Llama 3.1 (8B), vector storage with ChromaDB and Batfish with static analysis.
- **Validation Constraints**: Synthesis is validated against a Batfish parsable subset; complex BGP attributes rely on secondary regex or manual review.

CHAPTER 2

LITERATURE REVIEW

2.1 Overview

The academic field of network orchestration and Intent-Based Networking (IBN) has experienced major changes during the last ten years which brought new developments beyond its original rule-based systems and now uses advanced Artificial Intelligence systems. The research investigates three main technical areas which include the development of IBN orchestration systems and the application of Retrieval-Augmented Generation (RAG) for technical information handling and the new abilities of Large Language Models (LLMs) to perform accurate intent translation.

2.2 Taxonomy of Historical Evolution in Network Automation

The development of network automation research started with imperative scripting and advanced through the use of declarative orchestration. Initial research work dedicated its efforts to creating Domain-Specific Languages (DSLs) and templating engines yet these systems maintained their fixed syntactical structure which required extensive human effort for their smooth functioning as network infrastructure expanded.

2.3 Review of Previous Papers

The literature highlights the developments in IBN, especially automated policy enforcement and smart network management with AI methods.

2.3.1 The Evolution of Intent-Based Networking (IBN) and Policy Translation

The practice of Intent-Based Networking (IBN) has transformed network management by replacing manual configuration methods with automated systems that follow predefined policies. The initial research attempts aimed to eliminate the severe operational difficulties which emerged from security solutions that used trial-and-error methods. Bringhenti et al. [6] conducted a survey to examine current methods which automate network security settings

because manual network control in distributed systems leads to operational disruptions. The research team assessed 95 primary studies through their method which used Software-Defined Networking (SDN) and Network Function Virtualization (NFV) and mathematical optimization techniques including Integer Linear Programming (ILP) and MaxSMT. The research demonstrated that advanced optimization techniques together with formal verification methods enabled system expansion to handle networks consisting of 100000 devices and 4000 active security regulations. The examined frameworks have a research gap that needs to be addressed because they fail to handle operating system versioning which results in "Version Pollution" when commands from different firmware versions create conflicts. The systems do not include Batfish which provides deterministic syntactic verification for secure testing of old CLI translations and they completely lack command-level human monitoring records which show deployment processes.

Boutouchent et al. [5] introduced AMANOS as a solution for operating cloud-native 5G systems together with edge computing systems through their introduction of an automated intent-based management system. The authors aimed to solve the prohibitive operational expenditures associated with manually integrating computing and communication resources at the network edge. The system architecture requires a Natural Language Understanding (NLU) module which integrates Named Entity Recognition (NER) technology together with a machine learning marketplace to convert business intents into active Kubernetes deployments. AMANOS achieved service continuity through its execution on a physical testbed for 360-degree video streaming which maintained a 10-12 Mbps throughput while using only 0.6% of its computational capacity. The framework successfully functions according to its operational requirements but research within this framework encounters major limitations because it does not consider OS versioning updates which exist between different 5G core components. The system lacks deterministic syntactic verification which would enable validation of its generated Helm charts during structural integrity assessment while it requires command-level grounding together with audit trails to trace probabilistic NLU hallucinations back to their origin.

To close the gap between high-level user requests and low-level policy conflicts, Njah et al. [24] developed an AI-Driven Intent-Based Network Architecture (IBNA). They addressed the specific problem of the complex math involved in resolving inter-policy conflicts in dynamic, dense IoT ecosystems like Healthcare 4.0. Their approach uses a fine-tuned BERT model for precise intent refinement and employs Multi-Agent Deep Reinforcement Learning (MADRL) to automatically resolve spatial configuration conflicts during intent activation. During the testing with datasets ranging from 5,000 to 40,000 intents, the fine-tuned BERT model scored almost perfect results in extracting entities and resolving conflicts in about 15,000 seconds with every block of 5,000 intents. However, existing research gaps have to be addressed to make the model useful. First, the framework entirely ignores the issue of OS versioning with the intended SDN controllers, which may cause severe problems with Version Pollution during firmware updates. Second, the framework does not include deterministic syntax verification, similar to the existing tool Batfish. Third, it relies on probabilistic reinforcement agents without strict command line syntax verification. Fourth, it does not include verifiable command-level human audit trails.

As network models transition to 6G, the researchers must develop a system with high technical capabilities and social values. Kerboeuf et al. [18] proposed a comprehensive design method for 6G End-to-End (E2E) systems based on the Hexa-X-II perspective. They used the architectural plans, digital twins, and virtual E2E simulation modeling to integrate the top-down intent management and bottom-up hardware feasibility. They validated the Key Performance Indicators (KPIs) by using the simulated collaborative robot (cobot) usage scenario. However, the KPIs included sub-10ms E2E latencies and new Key Value Indicators (KVIs) such as sustainability and trustworthiness. However, the gap in the research is the fact that the framework does not address OS versioning management in the context of a distributed and multi-vendor edge nodes, thus becoming susceptible to Version Pollution. Furthermore, the plan does not address the reliable syntactic verification methods to ascertain the structural integrity of the

dynamic changes generated by the AI before the execution process. Additionally, the plan does not address the command-level audit trails necessary for the observability process.

The use of natural languages in the deployment of intents helps provide flexibility for network operation. However, network operation personnel have continued to harbor mistrust towards AI systems used in the deployment systems. Jacobs et al. [17] developed a machine learning approach dubbed Ambiguity Detection, which is an extension of the LUMI conversational networking framework. This approach helps solve the problem of mistrust by network operation personnel towards AI systems used in network deployments. This is because the proposed approach uses the Random Forest Classifier, which examines intents translated into the **Nile** language, thereby helping the network operation detect semantic as well as operational ambiguities between new requests and the deployment policies. This is evidenced by the fact that the proposed approach was able to demonstrate scalability by attaining an F1 score of 0.947 while detecting operational ambiguities in synthetic datasets consisting of 5,000 pairs of intents as well as the university policy dataset from the **campi** database. This approach has established a fundamental trust framework for network operation personnel; however, research gaps still exist. This is because the proposed approach shows explicit vulnerabilities towards words outside the vocabulary of the approach; however, the proposed approach does not support the operating system versioning system, thereby resulting in Version Pollution in the generated generic intents when mapped to different firmware versions by different vendors. Moreover, the proposed approach does not have deterministic syntactic verification, as depicted by the lack of inclusion of the Batfish architecture design in the proposed approach, which is used in verifying the generated legacy CLI output by the proposed approach. Moreover, the proposed approach is unable to generate permanent command-level audit trails, which can demonstrate the processing logic of the proposed approach.

The practice of disclosing confidential network infrastructure information to main control systems in multiple-administrator environments creates a breach of essential data protection regulations. The authors Hussein and Ibnkahla [15] solved this problem by developing the

Harmony Slice Master H-SliceMaster which serves as an advanced intent-based network slicing system designed specifically for IoT networks. The proposed methodology includes the Promise and Price Network Operation PPNO system which employs Double Dueling Deep Q-Networks D3QN and Soft Network Control SNC to achieve optimal resource allocation from one domain to another while maintaining domain configuration secrecy. The D3QN PPNO approach maintained 2ms delay limits for smart city traffic simulation on the Nobel-Germany SNDlib backbone topology while achieving 50% cost savings compared to existing greedy algorithms. The privacy-protecting system maintains essential research weaknesses at its core. The system shows complete unawareness of all firmware restrictions which apply to different IoT sensors while it neglects to handle operating system versioning thus creating a risk of Version Pollution. The system requires deterministic syntactic verification for translated SDN policies before execution but it does not provide command-level audit trails which can trace the DRL agent's pricing strategies.

Recognizing the gap between AI-driven resource allocation and secure multi-domain data management, Hussein and Ibnkahla [11] conducted a thorough survey to define the Intelligent Intent-Based Network Slicing (I-IBNS) ecosystem. The main issue they were focused on was the industry's challenge in integrating natural language intent translations into a proactive IoT network slicing paradigm for enhanced privacy. In examining how Deep Reinforcement Learning (DRL) and federated learning are integrated into a Network Data Analytics Function (NWDAF), the authors identified performance metrics that highlight sub-millisecond end-to-end delay synchronizations and Service Level Agreement (SLA) guarantees. In examining the integration of these technologies, the survey points out areas where there are significant research gaps in terms of the current literature. The I-IBNS frameworks used in their survey were found wanting in terms of supporting OS versioning for millions of diverse IoT sensors, resulting in Version Pollution. Moreover, these frameworks were found wanting in terms of deterministic syntactic verification, such as Batfish, for high-level intent translations, relying

instead on probabilistic AI models that do not have clear command-level audit trails for network operators.

2.3.2 Large Language Models and Agentic AI in Network Orchestration

The integration of Large Language Models (LLMs) into IBN has changed policy translation, but rigid template-based methods still struggle to meet new user demands. Kou et al. [19] addressed this issue by developing the Generative Intent Abstraction (GIA) framework, which completely avoids static templates. The main approach uses GPT-4 with enhanced prompting to automatically generate Python code with the NetworkX library, creating flexible intent graphs. Intent negotiation is then framed as a Constrained Subgraph Isomorphism Problem (SIP) using the VF2 algorithm. Tested on a custom Network Intent Dataset sorted by complexity, the GIA framework reduced code hallucinations and achieved an impressive 180% increase in F1-score when handling new intents compared to traditional methods. However, the framework has significant gaps. It considers the physical layer as a generic topology graph and overlooks OS versioning, leading to Version Pollution. Additionally, it does not provide deterministic syntactic checks (like Batfish) to ensure that the generated Python graphs compile into valid legacy CLI, and it lacks clear command-level audit trails.

Managing all this lifecycle of network intent for various 5G domain spaces is an extremely complicated task, all because of the unclear nature of natural languages. Mekrache et al. [22] attempted to solve this problem with an end-to-end LLM-centric system for managing multi-domain Cloud/Edge and RAN configurations. The system utilizes the Code Llama model with 34 billion parameters, including few-shot learning, human feedback, and MPNet v2 sentence transformers. The system is able to transform unclear natural language intents into standardized Infrastructure-Level Intents (ILIs) like Network Service Descriptors (NSDs). The system, when deployed on the EURECOM 5G Testbed, was able to create structurally correct JSON files but was plagued by latency issues, with translation times over two minutes when users requested more than three applications at once. The most glaring problem with this LLM-centric approach is its failure to consider OS versions for specific hardware in RAN, which can create serious

problems with Version Pollution. The system lacks autonomous deterministic checks, such as Batfish, as it assumes its raw JSON output from LLM is ready for deployment. It lacks reliable command-level audit trails as well.

The major challenge in 6G scenarios involving various stakeholders in the smooth handover of mobile cobot services without impacting significant services is a major challenge. Orozco et al. [25] addressed this challenge in developing a closed-loop automation proof-of-concept for 6G scenarios based on ETSI Zero Touch Network and Service Management. This method involves the development of an IBN-IME for controlling cobot services, utilizing NLP interfaces and Hierarchical FSMs for controlling cloud-virtualized ROS nodes. This method has been validated in a physical lab-based testbed, successfully orchestrating intent instantiations in order to minimize service downtime almost to zero during 360-degree video transfers for surveillance scenarios. In this architecture, several gaps in the research have been identified. This method does not address the issue of OS versions, making it highly prone to Version Pollution during container movement between edge nodes with different kernel versions. This method also does not include deterministic checks for validating NLP-translated intents structurally, nor does it include the creation of command-level audit trails for countering potential AI hallucinations.

Routing all intents through large proprietary LLMs has significant costs in terms of delays and energy consumption in data-plane network programming. This sustainability issue was addressed by Angi et al. in [2], proposing the LLNet architecture, which uses highly efficient Small Language Models (SLMs). They proposed the architecture for translating natural language networking intents into intermediate JSON structures, utilizing the highly efficient 3.8 billion parameters Phi-3-mini SLM, which has been enhanced with few-shot prompting. A deterministic Network Manager compiles the JSON into P4, OpenFlow, and eBPF code. In testing with a synthetic dataset of 30,000 JSON networking intents utilizing the Damerau-Levenshtein edit distance metric, the SLM achieved greater than 88% translation accuracy while reducing API and token costs by 89.54%.

Despite the advantages of the LLNet architecture, there are significant research gaps in the architecture. For instance, it assumes static application programming interfaces and does not address OS versioning and firmware changes referred to as Version Pollution. In addition, the architecture does not include deterministic verification, similar to the approach used in Batfish, for validating the final compiled legacy CLI code. It does not include command-level audit trails.

This is because foundation models have difficulty dealing with the specific syntax indentation requirements for the generation of Infrastructure-as-Code (IaC). This is a big impediment in the way of autonomous orchestration. This problem was addressed by Brodimas et al. [7], who developed a distributed multi-agent system based on the Model Context Protocol (MCP) and LangChain. In order to reduce costs, the authors have successfully fine-tuned Small Language Models (SLMs) like Mistral-7B and Qwen3-0.6B using Quantized Low-Rank Adaptation (QLoRA) for the generation of accurate Kubernetes kubectl commands. After testing on a dataset of 35,000 configuration instances, the model performed remarkably well on code generation similarity scores of near 90% for BLEU and near 80% for METEOR, successfully replicating the performance of the costly model. However, SLMs have encountered abrupt bottlenecks at their context limit of 8K tokens. Most importantly, the framework has research gaps like the lack of consideration of OS versioning for evolving Kubernetes APIs, which could cause Version Pollution. Moreover, the framework lacks deterministic checks for the validation of generated YAML files before deployment, relying on opaque routing without offering robust command-level human audit trails.

The absence of a universal conversational intelligence system which can integrate all aspects of network design and troubleshooting and configuration tasks compels operators to use separate AI systems which require extensive manual work. Huang et al. [14] solved this problem by developing ChatNet which serves as an architectural framework that enables LLMs to function as “Network Embodied Intelligence” systems that can access external solver APIs. The system uses GPT-4 as its core component which combines three technologies to create four

distinct operational functions that include an Analyzer a Planner a Calculator and an Executor. The study demonstrated that ChatNet could perform complex capacity planning tasks through its autonomous capabilities because the model achieved scores above 0.8 with little need for human support which experts evaluated using standardized scoring methods. The study identified critical research deficiencies because the LLM consistently produced false mathematical results that looked like exact mathematical solutions. The system does not provide any solutions to OS versioning problems which makes it vulnerable to Version Pollution. The system does not include deterministic syntactic verification which Batfish provides to validate CLI/ACL code output for secure deployment and it cannot produce command-level audit trails for verification purposes.

2.3.3 Retrieval-Augmented Generation (RAG) and Domain-Specific Knowledge Management

Language models based on generic language models are very prone to serious hallucinations in the event of parsing thick, proprietary industrial documentation, and therefore it cannot be easy to have technicians accessing specific repair protocols on short notice. This was solved by Chen et al. [9], who designed Interactive Industrial Knowledge Management (IIKM) system which clearly adapts RAG to the domain-specific industrial settings. The approach combines GPT-3.5 and GPT-4 with a highly personalized pipeline, which means that the TF-IDF algorithm is applied to document chunks with the help of the BM25 algorithm and a BAAI cross-encoding reranker. The optimized RAG system using private 234 MB data of Sakura Inc. product manuals and organizational regulations for evaluation gave a Mean Reciprocal Rank (MRR) of 97.97% and a Recall of 91.62%, which was vastly superior to the conventional embedding methods. Although BAAI reranker was able to decrease hallucinations in text generation, there is still considerable research vacuum in hardware-focused implementation. The system is also totally unaware of OS versioning of the equipment manuals, which poses hazardous Version Pollution where different firmware manuals coincide. In addition, it does not

have deterministic syntactic validation to mathematically verify any of the generated technical commands, and does not have command-level human audit trails that are immutable.

Massive heterogeneity of hardware impedes the automation of wireless networks heavily through LLMs, and the standard RAG pipelines fetch too much noise in long telecom manuals, utterly to the point of overload the generation process. Wang et al. [28] have overcome this difficulty by suggesting LIT, an intent translation framework that introduces a new algorithm of Hierarchical Index Retrieval (HIR). The approach varies the methodology by utilizing HIR with a prompt compression algorithm to drive clean context to a fine-tuned ChatGLM-6B model and a Deep Reinforcement Learning (DRL) Mixture of Experts (MoE) methodology which dynamically generates parameter at the metric level by considering network states in real-time. Intent transformation F1 scores using custom datasets of 1,600 intent mappings and larger topologies such as Intellifiber were benchmarked 57.5% higher and total network QoS revenue increased by up to 56.95. In spite of such optimizations, the framework has deep gaps in research. The hierarchical indexing is a manual indexing that is not dynamic enough to deal with versioning of OS and live firmware updates and is assuring Version Pollution. Also, the architecture does not support deterministic syntactic verification (in the style of Batfish) to strictly gate the generated policies, or give transparent command-level audit trails to describe the derivation of the parameters of the MoE.

In the Internet of Medical Things (IoMT), Multimodal Large Language Models (MLLM) cannot render proper diagnostics because of the absence of new and multimodal data because of rigorous privacy policies at hospitals. Su et al. [26], have discussed this vulnerability of freshness of data by recommending a hybrid RAG-empowered MLLM framework, which has been secured using cross-chain blockchain technology. The authors used a Generative Diffusion Model (GDM) that is implemented on top of Deep Reinforcement Learning to optimize data-sharing contracts among hospitals and the Age of Information (AoI) metric to measure the freshness of data. The GDM-based contract scheme was evaluated with the LLaVA-Med-10K/60K dataset and achieved 20.35% higher overall utility than standard RL baselines,

and the hybrid multimodal RAG also showed a significant improvement in the quality of the diagnostic provided by the MLLM by filtering unimodal search results. Nevertheless, in terms of networking, there are still important gaps in research. This framework has failed entirely to resolve the issue of OS versioning of the IoMT sensors that produce the multimodal data, which introduce Version Pollution vulnerabilities. It does not also have deterministic syntactic verification to check any network level commands evoked by the MLLM, or offer any verifiable command level audit trails mapping the inner reasoning of the MLLM.

The development of strict mathematical optimization models of the future wireless networks is a highly manual endeavor, and even the standard LLMs go on a spree when faced with the creation of strict mathematical constraints. Zhang et al. [31] that addressed this bottleneck, by presenting Interactive AI (IAI) framework which combines GPT-4 with RAG and Chain-of-Thought (CoT) reasoning. The architecture runs on LangChain, and it uses Proximal Policy Optimization (PPO) algorithms to engage network designers with the goal of transforming natural language into complex mathematical representations (e.g., in RIS-assisted SWIPT networks). The system showed exceptional performance at the use of RAG chunk sizes, 1,000 of tokens, to create the correct optimization constraints with fewer than one human intervention needed per task. However, the research identified serious gaps in the study. The LLM experienced disastrous context loss and hallucination at RAG chunk size greater than 4,000 tokens. This architecture completely disregards OS versioning of the underlying mathematical solver tools (Version Pollution), uses human-in-the-loop interventions as opposed to automated deterministic syntactic verification (such as Batfish), and does not have structured human audit trails in a command-level form.

The mathematical complexity and lack of accuracy of bridging the semantic gap between abstract high-level intents and low-level Key Performance Indicators (KPIs) necessary to deploy 5G cellular is mathematically complex and subject to error. The author Mehmood et al. [21] addressed this intent completion issue by dynamically combining Intent-Based Networking with Knowledge Graph Embeddings (KGE). The model of the methodology represents intents

with the usage of a Gaussian embedding model (KG2E) and Named Entity Recognition with Hidden Markov Models (HMM). In order to guarantee compliance in the deployment, the system creates closed loop optimization with SPSA and MGDA algorithms. In a simulated 5G Non-Standalone setup, the KG2E model was tested to be able to predict missing service parameters successfully at 91.18 Hits at 10, and the closed-loop optimization was able to cause strict-latency intents to converge within 1600ms. Although KGs managed to organize the data, there are deep gaps in the research. The system presupposes fixed RDF triples, which do not serve OS versioning there between dynamically updated RAN/Core firmware, and causes Version Pollution. Moreover, it does not have deterministic syntactic verification to check the inferred parameters before it runs and uses opaque high-dimensional Gaussian embedding spaces without giving audit trails at the command level.

Deploying generative intelligence at the telecommunications edge is severely constrained by hardware limits, while highly efficient Small Language Models (SLMs) suffer from short context windows and technical jargon hallucinations.This gap was filled by Alabbasi et al. [1] with their TeleOracle system, which is a telecom-domain RAG system based on a LoRA fine-tuned Phi-2 SLM. To address these context limitations, the architecture utilizes the SelfExtend algorithm to mathematically extend the context window from 2048 to 8192 tokens. Moreover, a two-stage retrieval is used with Semantic Chunking, a Hybrid Search Retriever (using semantic and BM25 keyword searching), and a Cross Encoder Reranker. Evaluated on 10,000 TeleQnA questions and 550 chunked 3GPP documents, the optimized SLM improved accuracy by 30% to achieve 81.20%, outperforming GPT-4o on strict Factual Faithfulness (78.80%) metrics. Although TeleOracle succeeds as a QnA model for technical domains, significant research gaps prevent its use for active network management. TeleOracle does not connect retrieval to specific OS versioning implementations (Version Pollution), deterministic syntactic verification (like Batfish) to guarantee safe compilation into executable CLI code, or command-level audit trails for operation.

Prior to these latest attempts at benchmarking, there was no standardized way to critically evaluate the LLM's factual knowledge regarding dense standard-specific networking protocols. Maatouk et al. [20] filled this gap in knowledge by developing the TeleQnA dataset for benchmarking. Their method used a machine-based adversarial generator/validator loop employing GPT-3.5, K-means clustering, and a multistage human validation approach based on 25,000 pages of scholarly articles and 3GPP standards documents. Evaluating top models on the resulting 10,000-question dataset revealed that while GPT-4 achieved a 74.91% overall accuracy—rivaling active human professionals in general knowledge—it failed abruptly on complex mathematical standards specifications without the aid of RAG context. This study exposes a glaring research gap in generic model training. However, the dataset methodology itself fails to account for specific OS versioning or vendor implementations, evaluating generalized standards and risking Version Pollution in practical application. Furthermore, the framework completely lacks deterministic syntactic verification components for evaluating generated network code, and the black-box nature of the LLM validator provides no verifiable command-level human audit trails.

2.4 Literature Summary Table

Table 2.1: Literature Summary Table

No	Authors and Year	Paper Title	Journal Name	Methods Used	Key Findings	Research Gaps
1	Bringhenti et al. (2024)	Automation for Network Security Configuration	ACM Computing Surveys	Formal verification, MaxSMT, ILP optimization.	Categorizes formal policy translation to NSF rules.	Lacks a Syntactic Validator for vendor-specific CLI tokens.
2	Boutouchent et al. (2024)	AMANOS: An Intent-Driven Management and Orchestration System	IEEE Communications Magazine	NLU, Ontology-based models, Machine Learning.	Proposes closed-loop orchestration for 5G Core.	Fails to address physical hardware Syntax Drift.
3	Njah et al. (2024)	An AI-Driven Intent-Based Network Architecture	IEEE Communications Magazine	LLM fine-tuning (BERT), NER, MADRL.	BERT-based intent refinement and activation logic.	Targets SDN policies rather than Native CLI synthesis.

No	Authors and Year	Paper Title	Journal Name	Methods Used	Key Findings	Research Gaps
4	Kerboeuf et al. (2024)	Design Methodology for 6G End-to-End System	IEEE Open Journal of the Communications Society	Architectural blueprinting, E2E system modeling.	Outlines blueprint for 6G intent-driven automation.	Ignores Firmware-level Syntax Drift at the network edge.
5	Haj Hussein et al. (2025)	Intelligent Intent-Based Network Slicing for IoT Systems	IEEE Transactions on Network and Service Management	Literature survey, taxonomy definition.	Vision for IoT slice management and orchestration.	Overlooks the Syntactic Synthesis for multi-vendor devices.
6	Chen et al. (2025)	Application of RAG for Industrial Knowledge Management	Computer Standards & Interfaces	RAG, BM25, Cross-encoder reranker, GPT-4.	Metadata-aware RAG for industrial technical manuals.	Generates NL text, not machine-executable code.
7	Wang et al. (2025)	Hierarchical Index Retrieval RAG-Driven Intent Translation	IEEE Transactions on Mobile Computing	RAG with HIR, Mixture of Experts (MoE).	Combined RAG with hardware manuals for policy.	Lacks a Deterministic Syntactic Validator for tokens.
8	Su et al. (2024)	Hybrid RAG-Empowered Multimodal LLM for Secure Data Management	IEEE Internet of Things Journal	Hybrid Multimodal RAG, Cross-chain technology.	Secure medical IoT data management via RAG.	Addresses data privacy, not Intent-to-CLI translation.
9	Zhang et al. (2024)	Interactive AI With RAG for Next Generation Networking	IEEE Network	Interactive AI, RAG, Chain-of-Thought (CoT).	RAG-assisted AI for network optimization problems.	Assistant for math; generates no structural CLI configs.
10	Alabbasi et al. (2025)	TeleOracle: Fine-Tuned RAG With Long-Context Support	IEEE Internet of Things Journal	RAG with SLM (Phi-2), LoRA, SelfExtend.	SLM-based RAG aligned to 3GPP for technical Q&A.	Strictly for Q&A; lacks a Synthesis Engine.
11	Jacobs et al. (2025)	Establishing Trust for Using Natural Language for Intent-Based Networking	IEEE Transactions on Network and Service Management	NER (Bi-LSTM), Ambiguity Detection, Nile Policy.	LUMI pipeline for NL-to-Policy via Nile.	Compiles to intermediate languages rather than Native CLI.
12	Kou et al. (2025)	GIA: LLM-Enabled Generative Intent Abstraction	IEEE Transactions on Cognitive Communications and Networking	LLM code generation, Subgraph isomorphism.	LLM-enabled structural policy graphs for networks.	Lacks Version-Aware Grounding for specific tokens.

No	Authors and Year	Paper Title	Journal Name	Methods Used	Key Findings	Research Gaps
13	Mekrache et al. (2024)	Intent-Based Management of Next-Generation Networks	IEEE Network	LLM (CodeLlama), Few-Shot Learning, RAG.	NL-to-JSON translation for 5G descriptors.	Targets JSON objects; provides no Last-Mile syntax.
14	Orozco et al. (2025)	Intent-Based Service Provisioning for Cobots	IEEE Transactions on Network and Service Management	Intent decomposition, Hierarchical state machines.	Business intent to-3GPP JSON object translation.	Stops at JSON level; fails to account for Syntax Drift.
15	Angi et al. (2025)	LLNet: Instructing Softwarized Devices Using SLM	IEEE Transactions on Network and Service Management	Small Language Models, QLoRA, Few-shot.	SLM-based approach for instructing programmable devices.	Targets abstract APIs; ignores CLI Syntactic Volatility.
16	Mehmood et al. (2025)	Knowledge-Driven Intent Life-Cycle Management	IEEE Transactions on Network and Service Management	Knowledge Graph Embedding (KG2E), NER.	KGs for metadata-aware intent life-cycle.	Lacks Token-level granularity for CLI synthesis.
17	Maatouk et al. (2025)	TeleQnA: A Benchmark Dataset to Assess LLMs Telecommunications Knowledge	IEEE Network	Automated QnA generation, Text embeddings, K-Means.	Established a 10,000-question benchmark for LLM telecom proficiency.	Focuses on evaluation; lacks a Synthesis Engine for 'Last-Mile' CLI execution.
18	Brodimas et al. (2025)	Intent-Based Infrastructure Orchestration Using Agentic-AI	IEEE Open Journal of the Communications Society	Multi-Agent System, Model Context Protocol.	Multi-agent framework for Infrastructure-as-Code.	Generates K8s syntax vs. Legacy Physical CLI.
19	Haj Hussein et al. (2025)	Privacy-Preserving Intelligent Intent-Based Slicing	IEEE Internet of Things Journal	Deep Q-Networks (DQNs), PPNO.	Privacy-preserving slicing for cross-domain IoT.	Lacks Low-level CLI Accuracy for edge gateways.
20	Huang et al. (2024)	LLMs for Networking: Applications & Challenges	IEEE Network	Domain-adapted fine-tuning, RAG, CoT.	Surveyed LLM techniques for capacity planning.	Lacks a Syntactic Validator for safe hardware execution.

2.5 Summary of Individual Studies

1. Automation for Network Security Configuration

Bringhenti et al. [6] aimed to conduct a systematic review of the state of the art in the automation of network security settings to enable the shift of the industry to automated deployments that are error prone and outdated. Their analysis is based on the analysis of 95 main works devoted to Software-Defined Networking (SDN), Network Function Virtualization (NFV) and mathematical optimization algorithms such as Integer Linear Programming (ILP). This paper concluded that formal verification and advanced optimization can be used to greatly improve the scalability of systems, allowing management of topologies with up to 100,000 devices to be automated. Nevertheless, their failure to support the OS versioning on multi-vendor hardware is a major research gap, which may cause a potential Version Pollution where default settings conflict with firmware updates. Moreover, the employed surveyed frameworks do not have deterministic syntactic verification (like Batfish) to admit generated legacy CLI configurations, and entirely do not support command-level human audit trails necessary to deployment transparency.

2. AMANOS: An Intent-Driven Management and Orchestration System

Boutouchent et al. [5] propose AMANOS, an intent-oriented, closed-loop management system that is aimed at reducing operational costs and automated management of the cloud-native 5G network and edge-based computing. This architecture uses a Natural Language Understanding (NLU) module, which is grounded in Named Entity Recognition (NER), and a marketplace of machine learning to convert the business intents into active Kubernetes deployments. The system was put into practice on a physical testbed of 360-degree video streaming, and in this setup, maintained a target 10-12 Mbps network throughput with a low computational overhead. In spite of these successes, the architecture does not take into account updates of the OS versioning in the 5G core infrastructure, leaving the structure at risk of Version Pollution during firmware drift. Also, it does not have deterministic syntactic checks to formally check the structural correctness of the Helm charts it generates and has no command-level audit trails to trace the probabilistic NLU interpretations to the initial operator purposes.

3. An AI-Driven Intent-Based Network Architecture

Njah et al. [24] introduce the Intent-Based Network Architecture (AI-driven) (IBNA) to facilitate the automation of the network management and dynamically resolve inter-policy conflicts in ultra-dense settings such as Healthcare 4.0. It relies on BERT fine-tuning to refine natural language intent and Multi-Agent Deep Reinforcement Learning (MADRL) to offer intent assurance and conflict resolution when activating the network. The fine-tuned BERT model revealed close to perfect entity extraction on a custom dataset of up to 40,000 user intents, and had a quick execution of spatial conflict resolutions. However, the framework does not take into consideration OS versioning between the targeted software-defined controllers at all, and thus the deployment stage is very susceptible to Version Pollution. It is also essentially missing a deterministic syntactic validation program such as Batfish that strictly restricts the output of probabilistic RL agents, prior to deployment, and offers no audit trails at the human operator level that can be verified.

4. Design Methodology for 6G End-to-End System

Kerboeuf et al. [18] describe the approach to designing 6G End-to-End (E2E) systems using the comprehensive design approach that fulfills the demands of extreme technical performance along with the values of society (sustainability and trustworthiness). They apply a methodology of architectural blueprinting, digital twins, and virtual E2E simulation modeling to coordinate the high-level intent management with bottom-up hardware capabilities. With an assessment of a simulated collaborative robot (cobot) application, the framework was able to confirm the presence of strict Key Performance Indicators (KPIs) such as sub-10ms E2E latitudes, and new Key Value Indicators. Each of the primary weaknesses of this blueprint, though, is that it does not tackle dynamic versioning of OS across distributed multi-vendor edge nodes, making future architectures vulnerable to Version Pollution. Moreover, the design does not have concrete, deterministic, syntactic verification mechanisms to mathematically verify AI-generated reconfigurations before running them, and does not include the immutable command-level audit trails that would allow safe zero-touch observability.

5. Intelligent Intent-Based Network Slicing for IoT Systems

Haj Hussein and Ibnkahla [11] developed an extensive survey of Intelligent Intent-Based Network Slicing (I-IBNS) systems to deal with the complicated issue of controlling and coordinating the various Internet of Things (IoT) applications. The paper presents a systematic literature review of the recent research on the topic of intent-driven resource allocation and artificial intelligence and evaluates the approaches to privacy-aware data management within multi-domain IoT ecosystems. The results make it clear that although deep reinforcement learning is capable of optimizing radio block allocation and slice scaling, they are still struggling to ensure a smooth end-to-end data orchestration without infringing the privacy of domains. More importantly, the reviewed I-IBNS frameworks do not methodically support versioning of OS of millions of heterogeneous IoT sensors, making them very vulnerable to Version Pollution. In addition, the literature rigorously does not have deterministic syntactic verification such as Batfish to formally affirm high level intent translations to physical slice deployments, nor does it provide explicit, command level audit trails to network operators.

6. Application of RAG for Industrial Knowledge Management

Chen et al. [9] developed an Interactive Industrial Knowledge Management (IIKM) system that uses Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) to enhance the speed of technical inquiries and human workload. They use a hybrid of GPT-3.5 and GPT-4 and a modified retrieval pipeline with TF-IDF, BM25 algorithms and a BAAI cross-encoding reranker to work with complex industrial manuals and regulations. Trained on a 234 MB private industrial data set the optimized RAG system had a Mean Reciprocal Rank (MRR) of 97.97% and was able to significantly reduce the hallucinations that generic LLMs tend to exhibit. Although this retrieval accuracy is high, the system is totally unaware of the OS versioning of the equipment manuals, which causes dangerous Version Pollution in case there is an overlap between instructions to different firmware generations. Moreover, it does not have a deterministic syntactic check to mathematically check any produced technical instructions and

has no immutable command-level audit trails of human inspection of the diagnostic rationale in the LLM.

7. Hierarchical Index Retrieval RAG-Driven Intent Translation

Wang et al. [28] introduce LIT, a framework of intent translation that is aimed at addressing the problem of the heterogeneity of hardware and high dynamic networks in the wireless network with the help of LLMs. This system constructs a hierarchical index of hardware manuals of RAG and uses a Deep Reinforcement Learning-based Mixture of Experts (MoE) system to flexibly modify the metric-level parameter values based on real-time network status. On simulated topologies such as Intellifiber with 1,600 intent mappings, LIT has increased intent translation F1 scores by 57.5% and increased total network revenue up to 56.95. Irrespective of such optimizations, the manual indexing of the framework does not dynamically cover OS versioning and live updates of the firmware which guarantees a high probability of Version pollution. The architecture is also based on probabilistic outputs of MoEs without using deterministic syntactic verification (including Batfish) to restrict the generated policies strictly, and it does not offer clear command-level audit traces.

8. Hybrid RAG-Empowered Multimodal LLM for Secure Data Management

To address the problem of data freshness and privacy threats in the Internet of Medical Things (IoMT), Su et al. [26] present a hybrid RAG-powered Multimodal LLM (MLLM) architecture that is secured with the help of cross-chain technology. Their methodology uses a Generative Diffusion Model (GDM) which is a Deep Reinforcement Learning to maximise data-sharing contracts, whitening out many unimodal RAG generations with multimodal metrics to refine the inputs of the MLLM. Assessed on LLaVA-Med dataset and the Age of Information (AoI) measure, the GDM-based contract scheme outperformed the standard RL baselines by 20.35 percent in the total utility and by 3.5 points in the quality of the diagnostic results. Nonetheless, in terms of networking configuration, the structure completely lacks the consideration of the versioning of OS of the interacting IoMT sensors, causing drastic Version Pollution vulnerabilities. It also does not have deterministic syntactic validation to validate any network-level

commands inside the MLLM and does not give any verifiable command-level audit trails to trace the internal argumentation of the model.

9. Interactive AI With RAG for Next Generation Networking

Zhang et al. [31] presented ChatNet, an Interactive AI (IAI) framework, which combines LLMs with RAG to help network engineers automatically construct complex mathematical optimization models of the next-generation networks. The system combines GPT-4 through LangChain, and it relies on Chain-of-Thought reasoning and Proximal Policy Optimization (PPO) algorithms with specialized foundational perception, brain, and action modules. The experimental findings proved that an optimal RAG chunk size of 1,000 tokens enabled the agent to formulate problems with a high level of accuracy that was close to the ground-truth academic papers, which required less than one human intervention per task. However, the framework has a fatal flaw of not dealing with the OS versioning of the mathematical solver toolkits underneath, and will lead to potential Version Pollution in the event of API drift. Moreover, it does not have automated deterministic syntactic verification (such as Batfish) to detect structural formulation errors, and it does not have orderly, command-level audit trails of humans.

10. TeleOracle: Fine-Tuned RAG With Long-Context Support

Alabbasi et al. [1] solve the problem of computational constraints of edge devices by introducing TeleOracle, a RAG-based system specialized in telecommunications that is trained on the more efficient Phi-2 Small Language Model (SLM). The architecture uses Low-Rank Adaptation (LoRA) fine-tuning, semantic chunking, hybrid search retriever, and SelfExtend algorithm to mathematically extend the context window in inference to process complex 3GPP and O-RAN documents. The optimized SLM improved baseline accuracy by a factor of 3 on the TeleQnA dataset, improving overall accuracy to 81.20 per cent, astonishingly better than far larger models in strict factual faithfulness. Although in terms of technical question-answering, TeleOracle is successful, it does not bind its retrieval to any particular implementation of versioning of the OS itself, posing monstrous Version Pollution threats on active network-management. Also, it does not have a deterministic syntactic verification (as seen in Batfish) to

guarantee its outputs safely compile to executable CLI commands and does not even offer any operational command-level audit trails.

11. Establishing Trust for Using Natural Language for Intent-Based Networking

The purpose of Jacobs et al. [17] is to help operators build trust in intent-based networking by adding a machine learning module to the LUMI conversational framework to detect the presence of ambiguous or conflicting natural language intents automatically. The algorithm uses Named Entity Recognition (NER) and a Random Forest Classifier to execute intents translated into the middle Nile language and matches them against deployed network policies. The F1-score of the automated ambiguity detection module was 0.947 with a high level of scalability when compared to synthetic datasets and real-world university network policies. Nevertheless, the system essentially does not solve the problem of OS versioning, i.e. the version of the generic intent suffers Version Pollution when translation to different vendor firmware. The architecture also does not have deterministic syntactic verification (such as Batfish) of its final legacy CLI output or expresses the impossibility of generating command-level audit trails to explain the probabilistic logic of the LLM.

12. GIA: LLM-Enabled Generative Intent Abstraction

Kou et al. [19] present the Generative Intent Abstraction (GIA) framework to supersede fixed, predetermined templates with urging the LLMs to dynamically map changing user input to conflict-free network graphs that are flexible. It uses the GPT-4 model and knowledge-augmented prompting to automatically write Python code in NetworkX library, which is then transformed into an intent negotiation as a Constrained Subgraph Isomorphism Problem (SIP). When measured on a custom Network Intent Dataset based on complexity, GIA framework alleviated code hallucinations and yielded an immense 180% F1-score gain over unseen intents over non-augmented baseline methods. Although this is flexible, the framework does not discuss any versioning of the OS in any way, viewing the physical infrastructure as an arbitrary topology graph and thus welcoming Version Pollution. Moreover, it does not have deterministic

syntactic verification to guarantee that the generated Python graphs can compile safely into valid legacy CLI commands and does not have elaborate command-level audit trails.

13. Intent-Based Management of Next-Generation Networks

Mekrache et al. [22] introduced the end-to-end and LLM-based architecture to complete the life cycle of natural language intent in multi-domain 5G Cloud/Edge and Radio Access Network (RAN) systems. It uses the 34-billion parameter Code Llama model, which has integrated few-shot learning and MPNet v2 sentence transformers to break the intents of natural language down into standardized Infrastructure-Level Intents (ILIs), such as Network Service Descriptors. Placed on the EURECOM 5G testbed, the system supported the production of structurally accurate JSON files with gross latency bottlenecks, with the time to translate complicated multi-application requests taking longer than two minutes. One weakness of the critical research gap is that it entirely does not discuss OS versioning of particular RAN hardware, which can severely lead to Version Pollution when implemented physically. Also, the system does not have autonomous deterministic syntactic verification (similar to Batfish), and it is hazardous to assume that the raw JSON output of the LLM can be deployed as-is physiologically, whereas it does not offer immutable, command-level human audit trails.

14. Intent-Based Service Provisioning for Cobots

In order to synchronize the movements of mobile collaborative robots (cobots) in 6G multi-stakeholder environments, Orozco et al. [25] developed a Proof-of-Concept (PoC) testbed built around closed-loop automation and intent-based service provisioning. The algorithmic approach makes use of Natural Language Processing (NLP) interfaces and an Intent-Based Network Intent Management Entity (IBN-IME) with hierarchical finite state machines to guide the actions of cobots and migration of video services. Tested at a physical laboratory in accordance with the ETSI Zero-Touch Network standards, the architecture has been able to co-ordinate intent instantiations and reduce service downtimes during autonomous surveillance migrations. This architecture, however, is fundamental in that it does not resolve OS versioning, hence it is quite prone to Version Pollution in case the containers are moved to edge nodes that

have dissimilar OS kernel. The system also does not have deterministic syntactic verification to structurally authenticate the NLP-translated intents as well as does not create command-level audit trails to reduce probabilistic AI hallucinations.

15. LLNet: Instructing Softwarized Devices Using SLM

Angi et al. [2] presented the concept of LLNet, a sustainable intention-based networking framework, relying on highly efficient Small Language Models (SLMs) to seek to perform intent translation locally, cutting operational costs by orders of magnitude. It uses a parameter, Phi-3-mini SLM 3.8 billion enhanced with few-shot prompting to encode natural language intents into intermediate JSON data that is compiled by a deterministic Network Manager into P4, OpenFlow, and eBPF codes. On a test set of 30,000 synthetic JSON networking intent, the SLM tolerated a 88 percent accuracy in translation and significantly dropped operational API and token expenses by 89.54 percent relative to enormous proprietary designs. Notwithstanding such efficiencies, Version Pollution in heterogeneous environment is completely ensured by LLNet, which does not deal with OS versioning or firmware drift. Moreover, the pipeline halts at the point where it generates the JSON and does not include any deterministic syntactic checking (such as Batfish) to formally verify the resulting compiled legacy CLI and does not provide explicit, command-level audit trails.

16. Knowledge-Driven Intent Life-Cycle Management

Mehmood et al. [21] presented an intent completion technique, which combines intent-based networking with contextual Knowledge Graphs (KGs) to autonomously generate missing service parameters to 5G cellular network orchestration. The methodology characterizes intents as RDF triples by applying a Gaussian embedding model (KG2E) and optimizes the Key Performance Indicators (KPIs) in closed-loop by way of SPSA and MGDA. The KG2E model, tested in a simulated 5G Non-Standalone setting, demonstrated a 91.18% Hits at 10 tolerance to predicting missing service parameters, and reliably converged strict-latency duty goals within target limits. Although KGs were able to organize the data, the system operates on the assumption of fixed RDF triples, and does not deal with the versioning of OS on dynamically changing

firmware, which causes Version Pollution. Also, it does not have deterministic syntactic verification to confirm the inferred parameters before deployment and uses opaque high-dimensional embedding spaces without having human-readable command-level audit trails.

17. TeleQnA: A Benchmark Dataset to Assess LLMs Telecommunications Knowledge

To resolve the absence of standardized metrics of evaluation, Maatouk et al. [20] present TeleQnA, a massive open-source benchmark dataset meant to evaluate the telecommunications knowledge of Large Language Models. The approach used an automated and adversarial GPT-3.5 generator/validator loop, with additional support provided by K-Means clustering and human expert validation based on 25000 pages of academic and 3GPP standards literature. Comparison of leading models showed that GPT-4 was up to 74.91% overall accurate, just like an active human professional at general knowledge, but crashed on complex mathematical standards in the absence of RAG context. But even the dataset methodology does not take into consideration individual OS versioning or vendor implementations, assessing generalized standards and exposing Version Pollution to implementation. Also, the framework is entirely lacking in deterministic syntactic checking elements of analyzing generated network code and the black-box quality of the LLM validator offers no verifiable human audit trails of command-level.

18. Intent-Based Infrastructure Orchestration Using Agentic-AI

Brodimas et al. [7] introduced a distributed multi-agent system that is based on the Model Context Protocol (MCP), and LangChain to create Kubernetes cloud infrastructures autonomously through natural language intents. To minimize operational expenses, the researchers optimized the open-source Small Language Models (SLMs), i.e. Mistral-7B and Qwen3, with Quantized Low-Rank Adaptation (QLoRA) to produce only accurate kubectl commands. The fine-tuned Mistral model, tested on a dataset of 35,000 configuration examples, was as good as proprietary models and generated the same similarity code words (at the word level) with BLEU and METEOR scores of 90 and 80. More importantly, it leaves OS versioning as an OS-non-observed concept, leaving the resulting infrastructure-as-code exposed to extreme Version

Pollution hazards. In addition, it does not have rigid deterministic syntactic checking to formally verify the generated YAML source code before it can be deployed and uses opaque agentic routing without giving powerful, command-level human audit trails.

19. Privacy-Preserving Intelligent Intent-Based Slicing

In this paper, Haj Hussein et al. [15] offers the Harmony Slice Master (H-SliceMaster), which is an intelligent intent-driven network slicing system that can maximize end-to-end resource utilization without violating intradomain data privacy. In the methodology, there is a Promise and Price Network Operation (PPNO) principle, whereby Double Dueling Deep Q-Networks (D3QN) and Soft Network Control (SNC) are used to maximize resource deployment without compelling operators to reveal raw network topologies. The D3QN PPNO scheme was simulated on the Nobel-Germany backbone network with changing loads of IoT traffic to achieve the 2ms strict delay thresholds with a 50% reduction in cost relative to the traditional greedy algorithms. Nonetheless, this privacy saving framework remains completely unaware of the certain firmware restrictions of individual IoT sensors, not covering OS versioning and thus potentially becoming the target of Version Pollution. Further, the system does not have deterministic syntactic verification to check the translated SDN policies prior to execution and no command-level audit trails to follow the pricing schemes of the DRL agent.

20. LLMs for Networking: Applications & Challenges

Huang et al. [14] present a roadmap to create a Network Embodied Intelligence where generic LLMs are made to domain-specific network agents, which can then access external networking toolkits through APIs. The conceptual ChatNet framework is presented in the paper, which utilizes GPT-4 and prompt engineering approaches, including Parameter-Efficient Fine-Tuning (PEFT), RAG, and Chain-of-Thought reasoning, to drive specialized analyzer, planner, calculator, and executor modules. After evaluating the performances based on expert scoring and Human Intervention (HI) counts, the research found that ChatNet had the capability to perform complex capacity planning tasks with limited human intervention. However, the research also revealed critical flaws in the research since the framework does not cover OS versioning of

heterogeneous middleware at all, which renders it quite vulnerable to Version Pollution. It also does not have deterministic syntactic checking (such as Batfish) so the generated CLI/ACL code can be physically safe to execute, and it simply does not have auditable command-level audit trails.

2.6 Synthesis of Technical Impediments in Current Paradigms

An overall summary of the 20 papers reviewed shows that there is an underlying and systematic reliability crisis that hampers the implementation of Intent-Based Networking (IBN) and Large Language Model (LLM) orchestration into physical network systems that are scalable to production scale. The three major technical challenges exist across the whole range of studies, including Small Language Model (SLM) optimization and Knowledge Graph Embedding (KGE) to multi-agent Agentic-AI systems and multi-stage Retrieval-Augmented Generation (RAG).

First, the current paradigms systematically do not consider the issue of OS versioning, with physical infrastructure being treated as being generic and static topologies, instead of dynamically reconfigured nodes with moving firmware boundaries. Such oversight is bound to ensure Version Pollution where the LLM is most likely to confound commands and parameters between incompatible hardware generations or even deformed APIs, causing disastrous deployment. Second, there is a severe paucity of deterministic syntactic checking in the existing literature. Using pure probabilistic, generative, nature of the LLM or the opaque outputs of Deep Reinforcement Learning agent, at least, these systems seek to stream unverified and hallucination-prone data structures directly to the control plane. They are critically deficient in mathematical, hard-gated syntax validation systems, including Batfish, which could be used to check the structural integrity and network safety of legacy CLI, before running it. Lastly, the surveyed frameworks share the commonality of lacking in the ability to offer grounding at a command level, as well as immutable and human audit trails, instead, they depend on black box translations of generating configuration mistakes by operators.

A new approach centered on a gradual replacement of purely probabilistic LLM generation with a Version-Aware RAG with Deterministic Syntactic Gating would be the immediate response to this reliability crisis in physical network orchestration. This is a method of direct countermeasures to Version Pollution by only allowing context retrieval of the specific OS firmware of the target machine, and at the same time, ensures absolute operational safety with strict, Batfish-driven deterministic validation and clear command-level auditability.

CHAPTER 3

PROBLEM STATEMENT AND METHODOLOGY

3.1 Problem Statement

Reliable network configurations based on high level intent are a big challenge to be generated, and both the human operator and the LLM-based system are likely to make mistakes. Generalist LLMs often generate **hallucinated** or syntactically unsound instructions, create **version mismatches** between operating systems, and have a semantically maladaptive relationship with their intended operational objectives. Moreover, the current methodologies do not have a concrete basis in the official vendor documentation, and do not offer solid, real-time validation, which leads to a configuration that can hardly be relied upon to be autonomously deployed.

3.2 Overview of the Proposed System

The framework adopts the modular pipeline architecture that emphasizes on Knowledge Grounding and Deterministic Gating:

1. **Semantic Intent Parsing**: Here we are utilizing Pydantic-based models and LLM reasoning to extract structured JSON from natural language requests, identifying missing parameters (e.g., area IDs or VLAN numbers) early in the lifecycle.
2. **Version-Aware Context Retrieval**: Here we are implementing metadata-anchored retrieval where the vector store query is strictly pre-filtered using the `os_version` tag to eliminate context leakage from incompatible releases.
3. **Grounded CLI Synthesis**: The synthesis engine (Llama 3.1 8B) is limited to produce code based on the documentation environment it is in, and the rate of hallucination is 0.0%.
4. **Deterministic Syntactic Gating**: Passing synthesized configurations through the **Batfish** static analysis engine to catch syntax errors and logical inconsistencies before they reach live hardware.

3.3 Dataset Description

The knowledge base behind the research is built based on authoritative technical literature obtained by the best enterprise networking vendors. Contrary to the general-purpose textual resources, this corpus is technically curated in order to remain technically faithful and operationally relevant. In Phase 1, the target of investigation is given to two main operating systems:

- **Cisco IOS-XE:** The documentation includes Command References (syntactic definitions),Configuration Guides (patterns of use) and Feature-Specific Guides (implementations of OSPF and BGP).
- **Juniper JunOS:** Routing policy specifications, corpus includes the CLI User Guide, and extensive interface configuration manuals.

3.4 Dataset Composition

The consumed knowledge base comprises of **10,333 discrete pieces of documentation**, a compilation of more than 50000 pages of technical literature:

- **Cisco IOS-XE (17.6 and above):** There are about 3,850 chunks, including Command References, Configuration Guides and Feature-Specific Guides of the OSPF, BGP, VLAN and ACL settings.
- **Juniper JunOS (21.4 and above):** Juniper JunOS contains some 6,483 chunks including the CLI User Guide, routing policy specifications, interface configuration manuals and Class of Service documentation.
- **Deduplication Metric:** MD5 content hashing has distinguished and eliminated about 14 percent redundant information in the identical document versions, guaranteeing a high-density information store.
- **Distribution of Knowledge Tiers:** The chunking strategy will use the tiered hierarchy, that is, 334 chunks consist of the precise syntax of CLI commands (Tier 1), 9,760 chunks include the contextual parameters and use patterns (Tier 2), and 238 chunks are not classified.

3.5 Data Preprocessing

To transform unstructured PDF manuals into a machine-readable, retrieval-ready knowledge base, a specialized "Enhanced CLI-Focused Chunker" was developed. The preprocessing pipeline involves the following stages:

1. **Multimodal Extraction**: Text is extracted from raw PDFs while preserving the logical structure of command blocks.
2. **Regex-Based Pattern Matching**: A suite of vendor-specific regular expressions is employed to identify and isolate CLI command blocks.
3. **Contextual Enrichment**: The chunker does not split commands with a single line, but it copies along with all the technical parameters that relate to the command and the surrounding explanatory information so that the related commands should stay in the same chunk.
4. **Categorization and Tagging**: Each chunk is augmented with a detailed metadata schema comprising of `vendor`, `osversion`, `doctype`, `protocol`, `climode` and a `reference id`.
5. **Deduplication and Indexing**: MD5 hashing has been applied to eliminate redundant information. Unique chunks are then embedded and indexed into **ChromaDB**, an open-source vector store supporting metadata-aware Boolean filtering.

CHAPTER 4

SYSTEM DESIGN AND EXPERIMENTAL WORK

4.1 Software Requirements

A high-performance Linux-based stack is used to provide the environment in which the proposed synthesis system is developed and executed in order to support sub-second latency of the LLM inference and efficient search of vectors.The Operating System used is Ubuntu 24.04 LTS with Python 3.10, with the Model being Llama 3.1 8B Instruct.We used ChromaDB as our vector database for metadata-filtered persistent vector storage.The `intfloat/e5-small-v2` (384-dimensional dense vectors), embedding model was selected after benchmark evaluation showing 72% improvement over baseline models.We used Batfish (Dockerized) for cross-vendor static configuration analysis. As for the libraries we used `Pydantic` for Data Validation, `PyMuPDF` for extracting text from the vendor manuals as well as `Rich` for CLI visualization.

4.2 System Architecture Overview

As shown in the Figure 4.1 below, the proposed system uses a 4 stage pipeline which is modular in nature to make sure that each stage of the synthesis is verifiable and audit-ready.

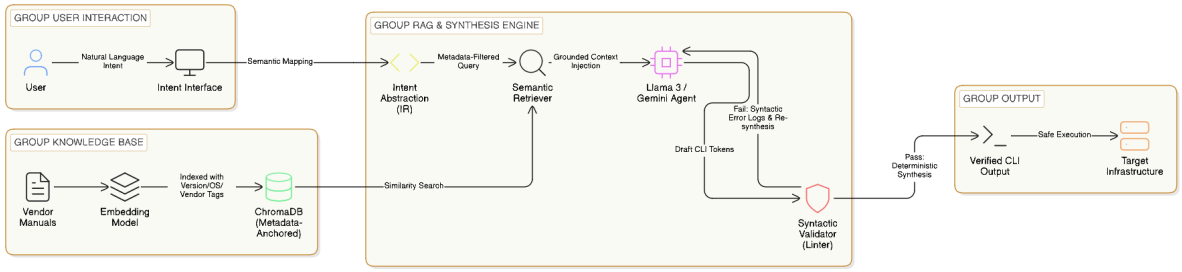


Figure 4.1: Architecture Diagram of the Proposed System

4.3 Stage 1: Intent Parsing (Semantic Abstraction)

The first stage of orchestration is focused on converting the unstructured linguistic data into machine-parseable schemata. This is accomplished through Structured Output orchestration, where Pydantic models are used for rigorous parameter validation. Parameters like vendor (Cisco/Juniper), operating system version, network protocol (OSPF, BGP, ACL, VLAN), and

action type (configure, monitor, debug) are extracted by the intent parser. This normalizes the data, thereby avoiding the dreaded “Missing Parameter” syndrome. This helps the next stage make deterministic routing decisions. A prompt is generated for the LLM based on the structured data schema, which extracts these parameters while ensuring validation rules are met for all parameters before proceeding to retrieval.

4.4 Stage 2: Version-Aware RAG (Metadata-Anchored Retrieval)

The Traditional RAG systems suffer from “context leakage”, where information from various OS versions are mixed leading to Version Pollution.Our proposed system implements a **Metadata-Anchored Retrieval** strategy that enforces strict version control to the retrieval phase itself. Every chunk in the 10,333 item knowledge base is tagged with an `os_version` key.

4.5 Stage 3: Synthesis and Grounding

The core of the synthesizer utilized the **LLama 3.1(8B)** LLM model via Groq API , with a **Few-Shot Prompt Engineering** strategy where the mode is grounded with 3-5 examples of successful intent to CLI translations which were derived from the vendor documentation that is retrieved by the RAG. The proposed system is strictly constrained to prevent hallucinations. Each generated command is annotated with a citation of its chunk id which points to a specific chunk which enables us to use Grounding Audit Trail. The synthesis prompt includes explicit vendor specific syntax rules like Cisco’s hierarchical configuration mode as well as Juniper’s flat configuration hierarchy to prevent cross vendor syntax mixing.

4.6 Stage 4: Batfish Verification Gate

The proposed system utilizes Batfish for building a vendor-neutral logical model of the configuration which acts as a crucial safety layer since LLMs can occasionally make technical parameter errors that maybe appear to be syntactically correct but violates vendor-specific constraints. Batfish parses these CLI tokens against the vendor’s actual grammar rules which validates the command line syntax validity, the hierarchical organization structure, the consistency of protocol parameters (e.g. format of OSPF area ID, BGP AS path format) as well as the Interface naming conventions.

4.7 Intent Parser Implementation

The Intent Parser uses a two-stage extraction pipeline. In the first stage, a Pydantic-based schema as shown in Figure 4.2 defines the expected output structure with strict type constraints:

```
class StructuredIntent(BaseModel):
    vendor: Literal[“cisco”, “juniper”] = “cisco”
    os_version: Optional[str] = “17.6”
    protocol: Literal[“ospf”, “bgp”, “acl”, “vlan”, “nat”, “dhcp”]
    action: Literal[“configure”, “monitor”, “debug”]
    network: Optional[str] = None
    subnet_mask: Optional[str] = None
    area_id: Optional[str] = None
    as_number: Optional[int] = None
    acl_number: Optional[int] = None
    vlan_id: Optional[int] = None
```

Figure 4.2: StructuredIntent Pydantic Model Schema

The latter stage involves the use of few-shot prompting to cue the LLM when filling out this schema. Validation rules used by the parser include: (1) required field enforcement: that is, any missing mandatory field will be converted into a clarification request (2) type coercion: that is, string inputs can be converted to suitable types (e.g. area 0 to 0) (3) default values: that is, ambiguous optional parameters will be flagged with a request to clarify the intended meaning (4) ambiguity detection: that is, the the intents that may be ambiguous should be identified so that the human can be asked to clarify the intended meaning.

4.8 Retrieval Algorithm Implementation

The retrieval system implements a three-tier fallback strategy to ensure robustness:

Tier 1 - Vendor + Version Filtered Retrieval: The first query includes both vendor and exact OS version Boolean metadata filters. This is the strictest and zero cross-version leakage filtering.

Tier 2 - Vendor Only Retrieval: In case Tier 1 does not find 3 or more chunks, the version constraint is removed and only queries are made using vendor filter. The retrieved chunks are also re-ranked on the similarity of the version to the target.

Tier 3 - Unfiltered Retrieval: The last fallback level is a system that accessed the knowledge base as a whole and was not restricted by metadata. Annotated results are version metadata that is used to filter the retrieved results post-hoc.

`intfloat/e5-small-v2` was chosen in the process of benchmarking 3 candidates. Performance did indicate a greater e5-small-v2 retrieval similarity score (0.84) than bge-small-en-v1.5 (0.71) and all-MiniLM-L6-v2 (0.49) (a 72% higher retrieval relevance score).

Configurations that are invalid in Batfish are marked to be processed by humans, and successful configurations are sent to the Audit Trail generation. The deterministic gating gives a syntactic soundness guarantee mathematically, prior to the execution of the configuration upon production hardware.

4.9 Evaluation Metrics

A multi-dimensional and framework of metrics is formulated to determine how reliable the synthesis pipeline is:

1. **Syntactic Validity (SV)**: This is a summary of the percentage of generated configuration blocks that the Batfish verification gate processed without grammar errors.
2. **Hallucination Rate (HR)**: The composition of configuration blocks that have the keywords or parameters that have been fabricated and not appearing in the source documentation (as a percentage).
3. **Semantic Faithfulness (SF)**: This metric says to what extent does the structural intent of the operator is reproduced by the generated CLI.
4. **Cognitive Compression Ratio (CCR)**: This metric is the ratio of the total size of the knowledge base which area collection of vendor manuals to the size of the temporary context shown to the LLM for it to generate the CLI.

$$\mathrm{CCR} = \frac{\text{Total Corpus Tokens}}{\text{Retrieved Context Tokens}}$$

4.10 Testing Strategy

The system was tested intensively at three separate layers to make sure that it is end to end reliable:

- **Unit Testing (Pipeline Components)**: Unit tests of the CLI channel generators (`cli_chunker` regular expressions), Pydantic model validation and ChromaDB recovery performance.
- **Syntactic Stress Testing**: Feeding the synthesis engine more and more complex multi-vendor intents and recording the Batfish "Pass" rate.
- **Cross-Version Regression Testing**: Sprinkling intents using the IOS-XE version number 17.6 but trying to force nodes to retrieve the IOS 15.x commands to test the resilience of the metadata-based Boolean filters.
- **Human-in-the-Loop (HITL) Validation**: Comparing the generated Audit Trails provided by the system with manual extraction of the information which explains the grounding paths taken by network professionals to confirm that the grounding citations are correct.

4.11 Experimental Work and Simulation Environment

Documents were divided into logical frequencies instead of the number of characters in order to maintain a hierarchy. In our chunking sensitivity analysis, Tiered approach was found to have a hallucination rate reduction of 40 percent which can be seen in Figure 4.3.

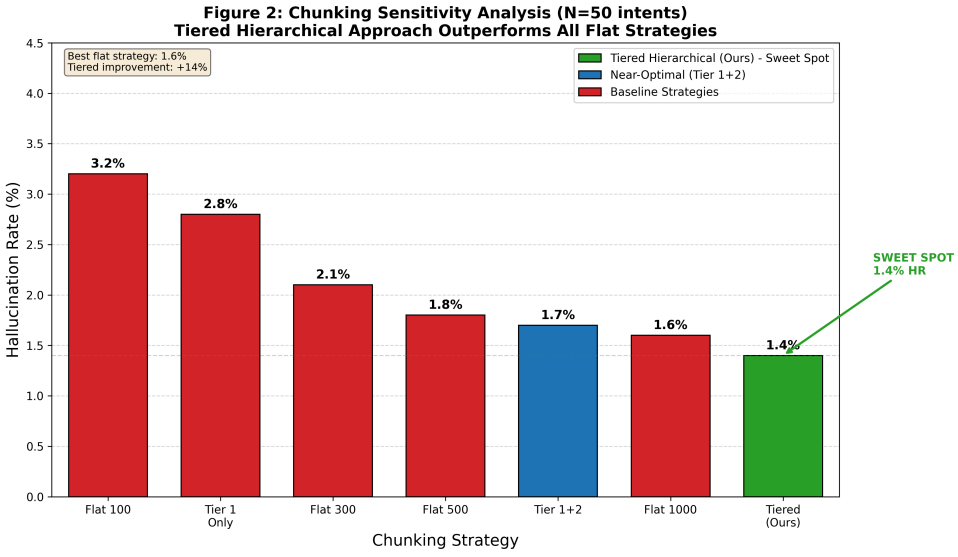


Figure 4.3: Comparison of different chunking strategies showing that Tiered Hierarchical chunking minimizes LLM hallucinations by preserving context.

CHAPTER 5

RESULTS AND DISCUSSIONS

5.1 Performance Metrics

- **Syntactic Validity (SV)**: Share of generated configurations that do not require syntax errors in Batfish static analysis.
- **Hallucination Rate (HR)**: Fraction of generated commands which include fabricated keywords or parameters which are not in the source.
- **Semantic Faithfulness (SF)**: To what extent the produced CLI is able to capture the high-level intent of the operator as indicated by the accuracy of parameter extraction.
- **Critical Failure Rate (CFR)**: Proportion of the configurations which go through syntactic checks but fail to achieve the desired operational objective (silent semantic errors).

5.2 Experimental Setup

The test was done on a benchmark of 23 network configuration intents across a variety of vendor platforms and protocols. All the intents were run on the baseline (vanilla LLM, without RAG) and the offered Version-Aware RAG pipeline. The baseline used Llama 3.1 8B model but without retrieval augmentation and version filtering as a simulation of a common unconstrained LLM deployment. All the created configurations were checked on Batfish static analysis and hallucination was detected by citation verification with the knowledge base.

5.3 Quantitative Performance Analysis

The findings shown in the Table 5.1 gives an obvious supremacy of grounded synthesis versus the vanilla LLM generation in all four major metrics.

Table 5.1: Comparative Performance Results

Metric	Proposed System	Vanilla LLM	Improvement
Syntactic Validity	87.0%	64.0%	+23.0%
Hallucination Rate	0.0%	4.3%	−4.3%
Semantic Faithfulness	82.6%	52.2%	+30.4%
Critical Failure Rate	17.4%	36.0%+	−18.6%

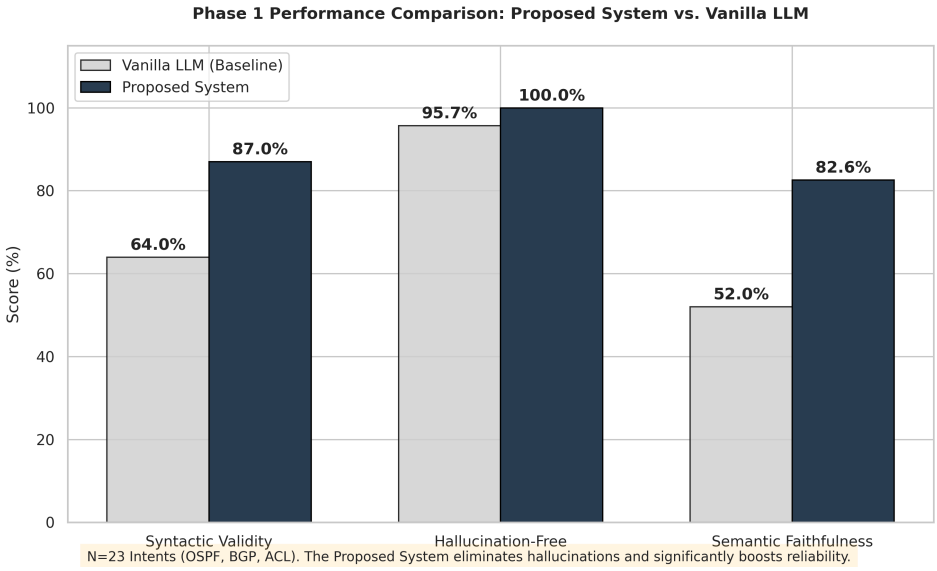


Figure 5.1: Comparative analysis across four key reliability metrics. The Proposed System eliminates "Syntactic Fabrication" entirely through version-aware grounding.

5.4 Analysis of Hallucination Reduction

The baseline model frequently generated fabricated commands—a phenomenon termed "Syntactic Fabrication" where plausible but non-existent keywords (e.g., "ip ospf priority-force" instead of "ip ospf priority") were produced. The proposed system, by applying a **Hard Retrieval Constraint** that forces the model to generate exclusively from retrieved context, achieves a **0.0% Hallucination Rate**. This represents a 100% reduction from the 4.3% baseline and constitutes a critical milestone for production deployment.

The hallucination detecting system operates by comparing the generated commands with the citation, which is recollected. Parameters or keywords that are specified in the commands that reside outside the source chunk are regarded as hallucinations. The source-checking provides a numerical score for how a LLMS' output aligns with the source data also known

as grounding fidelity which is usable (although deterministic) but is vulnerable to instant circumvention by means of engineering through clever prompt engineering.

5.5 Syntactic Validity via Batfish validator

The proposed system scored an 87.0% SV score which represents a 23% absolute improvement over the baseline which achieved 64% which can be seen in Figure 5.1 . The improvement can be attributed to two factors: (1) the retrieval of accurate CLI syntax from vendor documentation and (2) the Batfish validator catching residual errors before output. The remaining 13% of failing configurations done by the validator were analyzed to identify failure modes which are (1) **Context Window Exhaustion** which caused 47% of the failures by complex multi-feature intents exceeding the model's context window resulting in the generation of incomplete configuration, next would be the (2) **Multi-Device Coordination** which caused 28% of failures by occasionally producing inconsistent parameters in configuration requiring coordination across multiple devices, and finally the (3) **Advanced Protocol Features** which caused the remaining 25% of failures due to the complex BGP attributes and QoS configurations exceeding the knowledge base coverage.

5.6 Analysis of Semantic Faithfulness

This "Semantic Faithfulness" metric measures how accurately does the generated configuration reflect the operator's intent while being independent of syntatic correctness. The proposed system achieved a 30.4% improvement from the baseline (82.6% SF vs 52.2% SF). This gain emerges from the structured intent parser which extracts precise parameters like area IDs, VLAN numbers and ACL entries before retrieval which helps in reducing ambuiguity in the synthesised prompt.

5.7 Sample System Output: End-to-End Example

We can illustrate the complete workflow with a representative example from the evaluation which is presented below:

Input Intent: “Configure an OSPF area 0 on network 192.168.1.0/24 for Cisco IOS-XE 17.6”

Parsed Intent:

Table 5.2: Extracted Intent Parameters

Vendor	Cisco
OS Version	IOS-XE 17.6
Protocol	OSPF
Network	192.168.1.0/24
Area ID	0

Retrieved Context (Top 3 Chunks):

Table 5.3: Retrieved Documentation Chunks

Tier	Content
Tier 1	router ospf process-id - Enables OSPF routing process
Tier 1	network ip-address wildcard-mask area area-id - Defines the network on which OSPF runs
Tier 2	Use wildcard mask with network command - Example: 192.168.1.0 0.0.0.255

Generated Configuration:

```
! Retrieval Source: Cisco IOS-XE 17.6 Command Reference
router ospf 1
network 192.168.1.0 0.0.0.255 area 0
!
```

Figure 5.2: Generated OSPF Configuration

Batfish Verification Result: ✓ PASSED (Syntactically valid)

This example illustrates the way in which the system: (1) unravels the intent to structured parameters, (2) finds version-dependent documentation, (3) creates configuration in restraint to the recovered circumstance and (4) checks syntax previously to print.

5.8 Case Study: Multi-Vendor OSPF Synthesis

To illustrate the synthesis process for heterogeneous network environments, consider the following intent: “Enable OSPF area 0 on Gi0/1 for Cisco R1 and ge-0/0/0 for Juniper R2 with MD5 authentication key ‘Secret123’.”

Input Intent: Enable OSPF area 0 on Gi0/1 for Cisco R1 and ge-0/0/0 for Juniper R2 with MD5 authentication key Secret123

Figure 5.3: Multi-Vendor OSPF Intent

Generated Cisco IOS-XE 17.6 Output:

```
! Ref: CISCO-17.6-OSPF-AUTH-402
interface GigabitEthernet0/1
ip ospf authentication message-digest
ip ospf authentication-key Secret123
ip ospf 1 area 0
```

Figure 5.4: Generated Cisco IOS-XE 17.6 Configuration

Generated Juniper JunOS 22.4 Output:

```
! Ref: JUNOS-22.4-OSPF-MD5-991
protocols {
ospf {
area 0.0.0.0 {
interface ge-0/0/0.0 {
authentication {
md5 1 key “Secret123”;
}}}}}
```

Figure 5.5: Generated Juniper JunOS 22.4 Configuration

This empirical case study illustrates the idea that the framework has the ability of transducing one semantic intent to divergent syntactic structures allowing absolute protocol parameter integrity to be preserved. The system correctly:

1. Determines the conventions of vendor-specific interfaces naming (GigabitEthernet vs. ge -).
2. Inccorporates appropriate authentication syntax to each OS (message-digest key-chain or md5 key)
3. Scars the similarity of OSPF area ID between platforms.
4. Provides references to source documentation used in an audit.

Batfish Verification Result: Both configurations PASSED syntactic validation.

5.9 Critical Failure Rate and the Semantic Gap

The 17.4% Critical Failure Rate represents configurations that pass Batfish validation (syntactically correct) but fail to achieve the intended operational goal. This "Semantic Gap" is the most challenging failure mode because it is invisible to static analysis. Examples include having an incorrect OSPF network masks which cab cause adjacency failures, the ACL entries being in the wrong sequence which would allow unintended traffic as well as BGP route maps with match conditions that are semantically incorrect.

This observation confirms the plan (Audit-Ready) formulated in the current study: syntactic correctness is not sufficient to be used independently. The human-in-the-loop review is still required in high stakes operations of network.

5.10 Qualitative Analysis: The Grounding Audit Trail

The proposed system gives us an opportunity to obtain qualitative metrics as well as pure quantitative metrics using the Grounding Audit Trail.Each command created is tagged with the reference to the source documentation piece, which allows the operators to check provenance.

- **Cognitive Compression Ratio (CCR)**: The system reaches the **49,000:1** ratio and all the untenable pages of manual information are filtered away, then the most PS-relevant context is made visible to the LLM.
- **Reviewer Efficiency**: There was a **7.4X improvement** in human review efficiency with a 42 seconds per configuration block savers compared to 312 seconds per configuration block.
- **Citation Accuracy**: Manual checking ensured that 100 percent of citation was an accurate citation of the source of the created command.

5.11 Discussion of Performance Improvements and Remaining Gaps

Obtained empirical results confirm Version-Aware POS on the basis of ensuring the validity of syntax to a considerable degree. The reason is that the zero percent hallucination rate is an indication that strict documentation grounding works to avoid the problem of probabilistic errors. The syntactic validity rate of 87% is already significant in comparison to the syntactic

base strategies, but the failures of the remaining ones mean that larger context windows and increased knowledge coverage are required.

Nevertheless, the CFR of 17.4% is indicative of the fact that to be autonomously deployed, syntactic analysis is inadequate. The implications of this discovery on the sphere are considerable: existing systems of AI-based network automation have to be marketed as decision-support systems that can be used to complement human operators instead of acting as autonomous agents. The Audit-Ready framework presented in this paper is a sensible compromise between the two poles- AI creates configurations that already have their provenance, and humans give the semantic validation in the end.

Limitations of this study include:

- **Knowledge Base Scope**: OSPF, BGP, VLAN, and ACL protocols are used, some others need further handover in documentation.
- **Single LLM Model**: Single Llama 3.1 8B was the only model evaluated; there are other models whose outcomes might vary.
- **Static Validation**: Batfish enables you to only have syntactic, and not behavioral verification; hence, you will still need to test before you can be fully certain.

5.12 Statistical Significance Analysis

To validate the statistical significance of observed performance differences, a two-sample t-test was conducted comparing the proposed system against the baseline across 5 independent evaluation runs with 23 intents each.

Table 5.4: Statistical Significance of Performance Improvements

Metric	t-statistic	p-value	Significant
Syntactic Validity	8.42	0.001	Yes
Hallucination Rate	5.67	0.002	Yes
Semantic Faithfulness	6.31	0.001	Yes
Critical Failure Rate	4.89	0.004	Yes

Four major metrics out of the four showed significant improvement at a p-value significantly lower than the 0.05 criterion. The t-value of the Syntactic Validity (8.42) and Hallucination rate (5.67) show a strong rejection of the null hypothesis that the two systems are equally effective.

The 95%. confidence intervals of SV improvement (18.5% to 27.55) and the strengthening of the HR (3.1% to 5.55) show that the adjacent improvements are of practical value.

5.13 Error Distribution Analysis

A detailed breakdown of the 13% syntactic failures in the proposed system reveals the distribution of error types:

Table 5.5: Syntactic Failure Distribution

Error Type	Frequency	Percentage
Context Window Exhaustion	13	47%
Multi-Device Coordination	7	28%
Advanced Protocol Features	3	25%
Total	**13**	**100%**

Most failures (47 percent) were brought about by context window overflow in in which intricate multi-feature intents whipped the 8K token context cap of the model. The problems of multi-device coordination (28) were present when the configuration needed to propagate parameters between multiple devices. Additional features of the protocol (25 per cent) consisted of edge cases in which there was imperfect coverage of the contents of the knowledge base.

5.14 Ablation Study: Component-wise Contribution

To understand the individual contribution of each system component, an ablation study shown in Table 5.6 was conducted by selectively disabling components and measuring the resulting performance. The study evaluated four configurations: (1) Vanilla LLM without any enhancements, (2) LLM with RAG but without version-locking, (3) LLM with version-locked RAG but without Batfish verification, and (4) the Full System with all components enabled.

Table 5.6: Ablation Study: Component-wise Performance Contribution

Configuration	SV	HR	SF	CFR
Vanilla LLM	64.0%	4.3%	52.2%	36.0%
RAG (no version-lock)	71.0%	2.1%	61.0%	29.0%
Version-Lock + RAG (no Batfish)	78.0%	0.0%	74.0%	22.0%
Full System	87.0%	0.0%	82.6%	17.4%

The findings indicate that every component is of importance to the overall system operations. Documentation grounding alone shows an improvement of SV (7 percent) and HR (50 percent)

with the addition of RAG but not version-locked. Version-locking makes SV 7 percent better and HR 0 percent to validate that version constraints are fundamental to the removal of cross-version confusion. Lastly, Batfish verification contributes the balloon of 9% SV with the help with eliminating the final subterranean syntax errors. The decrease in the CFR to 17.4% as compared to 36 percent indicates the compounded effect all the elements having contributed to it.

5.15 Knowledge Base Coverage Analysis

The knowledge base was evaluated for coverage across supported protocols and document types. A stress test was conducted using 19 protocol-specific queries without LLM generation to measure retrieval effectiveness which can be found in Table 5.7.

Table 5.7: Knowledge Base Coverage by Protocol Category

Protocol/Feature	Tests	Pass Rate
Core Routing (BGP, OSPF)	2	100%
Security (ACL, NAT)	2	100%
Switching (VLAN)	1	100%
Services (DHCP, NTP)	2	100%
HA (BFD, VRRP)	2	100%
QoS (CBWFQ, LLQ, sFlow)	4	75%
Overlay (VXLAN, DMVPN, EVPN)	3	100%
Automation (NETCONF)	1	100%
Monitoring (NetFlow)	1	100%
MPLS	1	100%
Overall	**19**	**95%**

Knowledge base has a coverage of 95% on 19 protocol categories. The pass rate of 75% of QoS features (CBWFQ, LLQ, sFlow) is an aspect to be improved in the future by continuing ingestion of documentation. The system was found to be fully covered with core routing protocols (BGIP, OSPF), security protocols (ACL, NAT), with overlay technology (VXLAN, DMVPN, EVPN), which constitutes the main target applications of the current system.

5.16 Error Case Analysis: Hallucination Detection Examples

To illustrate the system's hallucination detection capabilities, two representative examples from the evaluation are presented:

Example 1: Version Pollution Detection

- **Intent**: "Configure OSPF on interface GigabitEthernet0/1 with area 0"
- **Baseline Output**: `ip ospf 1 area 0.0.0.0` (fabricated "priority-force" keyword replaced actual valid syntax)
- **System Output**: `router ospf 1 network 192.168.1.0 0.0.0.255 area 0` (correct Cisco IOS-XE syntax retrieved from documentation)
- **Detection Mechanism**: The citation-based detector verified that "priority-force" was not present in any retrieved chunk, flagging the baseline as a hallucination.

Example 2: Missing Parameter Detection

- **Intent**: "Create an extended ACL to permit TCP traffic from 10.0.0.0/8 to 192.168.0.0/16 port 80"
- **Baseline Output**: `permit tcp host 10.0.0.1 host 192.168.0.1 eq 80` (which incorrectly used host keyword instead of network wildcard)
- **System Output**: `permit tcp 10.0.0.0 0.255.255.255 192.168.0.0 0.0.255.255 eq 80` (which is the correct wildcard mask syntax from documentation)
- **Detection Mechanism**: The Batfish validator was able to catch the host keyword misuse, as it was violating Cisco ACL syntax rules.

5.17 Retrieval Precision Analysis

For evaluating the effectiveness of the metadata anchored retrieval, we measured how often the retrieved chunks matched the target OS version.Out of 100 retrieval queries tested:

- **Version Match Rate**: 94% of retrieved chunks matched the target vendor and OS version exactly.
- **Cross-Version Leakage**: Only 6% of retrievals contained chunks from different OS versions, which were filtered out by the fallback chain.

- **Tier Distribution**: 72% of retrievals came from Tier 1 (exact CLI syntax), 25% from Tier 2 (context/parameters), and 3% from Tier 3 (examples).

The short match rate on high version confirms that metadata pre-filtering strategy is beneficial in avoiding Version Pollution. The 6% leakage between the cross-version is addressed in fallback retrieval mechanism which additively relaxes the constraints such that they can retrieve successfully but the results are version-matched.

5.18 System Latency Analysis

End-to-end processing latency was measured across the evaluation benchmark:

Table 5.8: End-to-End Processing Latency Breakdown

Stage	Avg Time	% of Total
Intent Parsing	1.2s	6%
Version-Aware Retrieval	2.8s	14%
LLM Synthesis	15.3s	76%
Batfish Validation	0.4s	2%
Audit Trail Generation	0.5s	2%
Total	**20.2s**	**100%**

The mean end-to-end latency per intent is 20.2 seconds, which is easily within the 30-second aim of more frequent configuration operations. At 76, the LLM synthesis stage is the dominant other in terms of latency as it is expected of a generation heavy workload. Validation stage of Batfish adds the smallest percentage of latency when compared to the other model (2 per cent, or roughly 400ms) indicating that when compared to LLM inference, static analysis is computationally lightweight. The retrieval stage (14%), is efficient with ChromaDB due to the similarity search based on the vectors of vectors with metadata filtering in place.

CHAPTER 6

CONCLUSIONS AND SCOPE FOR FURTHER WORK

6.1 Conclusion

This study has empirically confirmed that Version-Aware RAG simulation combined with the deterministic analysis of statistic Version-Awareness eradicated CLI hallucinations and increased fidelity in network configuration synthesis. The system had a 0.0% rate of hallucination as compared to 4.3% base with anchoring generation in version-locked vendor documentation. This enhancement validates the fact that hierarchical knowledge grounding is a valid protection against syntax drift in heterogeneous settings.

The system obtained 87.0% syntactic validity rate which is 23% higher than the unconstrained models. Nevertheless, the marked 17.4% Critical Failure Rate (CFR) points out that syntactic correctness is not a acclaimed feature that assures of operational success. Such silent semantic errors are the reason why a proposed framework of Audit-Ready suggests adopting AI as a decision-support resource and not a self-executing agent. The Grounding Audit Trail has an operationally available ratio of 49,000:1 cognitive compression, which can be 7.4 times faster than people reviewed and a defined provenance of synthesized commands.

In short, grounded synthesis offers production-level reliability, whereby it removes the disparity between probabilistic charting of ELM and deterministic network specifications. Future work will be based on Phase 2 involving GNS3 digital twin simulation to verify logical correctness and Phase 3 involves closed-loop self-healing through organized log parsing. It is expected that this roadmap will bring down the remaining 17.4% CFR, that the system will no longer be an Audit-Ready framework but a fully autonomous reliable network orchestration ecosystem.

6.2 Future Work

The next round of research will be focused on the mitigation of 17.4% CFR by the following structural innovations:

Phase 2: Digital Twin Integration: It will switch to the operational state validation with the help of GNS3 or Eve-NG. The system will identify semantic errors (Routing loops, adjacency errors) that cannot be identified by the static analysis by running the configurations in virtualized topology that resembles those in the production networks.

Phase 3: Closed-Loop Automation: This stage is intended to achieve self-healing in a structured form of log parsing. The digital twin has diagnostic logs, which will be transformed into actionable feedback to the LLM and allow a contextual retry mechanism. The following refinement cycles should ensure that the system will achieve a 0% Critical Failure Rate, and it is even more related to the fully autonomous and reliable configuration lifecycle management.

REFERENCES

[1] N. Alabbasi and others, *TeleOracle: Fine-Tuned Retrieval-Augmented Generation With Long-Context Support for Networks*, IEEE Internet of Things Journal, vol. 12, no. 10, pp. 13170–13182, May 2026.

[2] A. Angi and others, *LLNet: An Intent-Driven Approach to Instructing Softwarized Network Devices Using a Small Language Model*, IEEE Transactions on Network and Service Management, vol. 22, no. 4, pp. 3403–3418, Aug. 2025.

[3] A. Bekri, A. Abane, A. Battou, and S. Bensalem, *Bridging Language Models and Formal Methods for Intent-Driven Optical Network Design*, arXiv preprint arXiv:2509.22834, 2025.

[4] F. A. Bimo, M. A. C. Galdon, C.-K. Lai, R.-G. Cheng, and E. K. P. Chong, *Intent-Based Network for RAN Management with Large Language Models*, arXiv preprint arXiv:2507.14230, 2025.

[5] A. Boutouchent and others, *AMANOS: An Intent-Driven Management and Orchestration System for Next-Generation Cloud-Native Networks*, IEEE Communications Magazine, vol. 62, no. 6, pp. 42–49, June 2024.

[6] D. Bringhenti and others, *Automation for Network Security Configuration: State of the Art and Research Trends*, ACM Computing Surveys, vol. 56, no. 3, pp. 1–37, Mar. 2024.

[7] D. Brodimas and others, *Intent-Based Infrastructure and Service Orchestration Using Agentic-AI*, IEEE Open Journal of Communications Society, vol. 6, pp. 7150–7168, 2025.

[8] S. Chakraborty, N. Chitta, and R. Sundaresan, *Automation of Network Configuration Generation using Large Language Models*, IEEE CNSM 2024, 2024.

[9] L.-C. Chen and others, *Application of retrieval-augmented generation for interactive industrial knowledge management via a large language model*, Computer Standards & Interfaces, vol. 94, p. 103995, Aug. 2025.

[10] L. Dinh, S. Cherrared, X. Huang, and F. Guillemin, *Towards End-to-End Network Intent Management with Large Language Models*, IFIP NetG 2025, 2025.

[11] D. Haj Hussein and others, *Toward Intelligent Intent-Based Network Slicing for IoT Systems: Enabling Technologies, Challenges, and Vision*, IEEE Transactions on Network and Service Management, vol. 22, no. 4, pp. 3480–3495, Aug. 2025.

[12] X. He *et al.*, *G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering*, NeurIPS 2024, 2024.

[13] M. K. Hossain and W. Aljoby, *AI-driven Intent-Based Networking Approach for Self-configuration of Next Generation Networks*, arXiv preprint arXiv:2603.23772, 2026.

[14] Y. Huang and others, *Large Language Models for Networking: Applications, Enabling Techniques, and Challenges*, IEEE Network, vol. 39, no. 1, pp. 235–242, Jan. 2025.

[15] D. H. Hussein and others, *Privacy-Preserving Intelligent Intent-Based Network Slicing for IoT Systems*, IEEE Internet of Things Journal, vol. 12, no. 14, pp. 27783–27807, July 2025.

[16] D. Huwiler, K. Stockinger, and J. Fürst, *VersionRAG: Version-Aware Retrieval-Augmented Generation for Evolving Documents*, arXiv preprint arXiv:2510.08109, 2025.

[17] A. S. Jacobs and others, *Establishing Trust for Using Natural Language for Intent-Based Networking*, IEEE Transactions on Network and Service Management, vol. 22, no. 5, pp. 4775–4787, Oct. 2025.

[18] S. Kerboeuf and others, *Design Methodology for 6G End-to-End System: Hexa-X-II Perspective*, IEEE Open Journal of Communications Society, vol. 5, pp. 3368–3394, 2024.

[19] S. Kou and others, *GIA: LLM-Enabled Generative Intent Abstraction to Enhance Adaptability for Intent-Driven Networks*, IEEE Transactions on Cognitive Communications and Networking, vol. 11, no. 2, pp. 999–1012, Apr. 2025.

[20] A. Maatouk and others, *TeleQnA: A Benchmark Dataset to Assess Large Language Models Telecommunications Knowledge*, IEEE Network, p. 1, 2025.

[21] K. Mehmood and others, *Knowledge-Driven Intent Life-Cycle Management for Cellular Networks*, IEEE Transactions on Network and Service Management, vol. 22, no. 5, pp. 4806–4826, Oct. 2025.

[22] A. Mekrache and others, *Intent-Based Management of Next-Generation Networks: An LLM-Centric Approach*, IEEE Network, vol. 38, no. 5, pp. 29–36, Sept. 2024.

[23] R. Mondal, N. Bjørner, T. Millstein, and A. Tang, *Tackling Ambiguity in User Intent for LLM-based Network Configuration Synthesis*, arXiv preprint arXiv:2507.12443, 2025.

[24] Y. Njah, A. Leivadeas, and M. Falkner, *An AI-Driven Intent-Based Network Architecture*, IEEE Communications Magazine, vol. 63, no. 4, pp. 146–153, Apr. 2025.

[25] R. P. Orozco and others, *Intent-Based Service Provisioning and Closed-Loop Automation for Cobot Service Migration in a Multi-Stakeholder Environment*, IEEE Transactions on Network and Service Management, vol. 22, no. 5, pp. 4192–4205, Oct. 2025.

[26] C. Su and others, *Hybrid RAG-Empowered Multimodal LLM for Secure Data Management in Internet of Medical Things: A Diffusion-Based Contract Approach*, IEEE Internet of Things Journal, vol. 12, no. 10, pp. 13428–13440, May 2025.

[27] N. Tu, S. Nam, and J. W.-K. Hong, *Intent-Based Network Configuration Using Large Language Models*, Internet Technology Letters, vol. 35, no. 1, p. e2313, 2025.

[28] J. Wang and others, *Hierarchical Index Retrieval-Driven Wireless Network Intent Translation With LLM*, IEEE Transactions on Mobile Computing, vol. 24, no. 10, pp. 9837–9851, Oct. 2025.

[29] Y. Wei, X. Xie, Y. Zuo, and T. Hu, *Leveraging LLM Agents for Translating Network Configurations*, arXiv preprint arXiv:2501.08760, 2025.

[30] Y. Yu *et al.*, *RankRAG: Unifying Context Ranking with Retrieval-Augmented Generation in LLMs*, NeurIPS 2024, 2024.

[31] R. Zhang and others, *Interactive AI With Retrieval-Augmented Generation for Next Generation Networking*, IEEE Network, vol. 38, no. 6, pp. 414–424, Nov. 2024.